S(M,T): A Unified Scoring Framework for LLM Routing with Provable Convergence and the Measurement Gap

Pranav Lakherwal

pranav@pranavlakherwal.com

March 2026

Abstract

We present S(M,T), a unified scoring framework for routing queries to optimal language model endpoints. The framework combines multiplicative gating, weighted compatibility functions, and Boltzmann cost penalties into a single equation: $S(M,T) = \left[\prod_j g_j(M,T)\right] \cdot \left[\sum_i w_i \cdot \phi_i(M,T)\right] \cdot e^{-\beta(T)\cdot \mathbf{H}}$. The equation form is not new: under a log transform and softmax selection it is a constrained conditional logit (McFadden, 1974) with one log-sum compatibility regressor; our contribution is the parameterization of that form for heterogeneous endpoint routing, its analysis, and the empirical program. We prove convergence under Hajek conditions with logarithmic cooling, establish $O(\sqrt{KT\log T})$ regret bounds via contextual bandits reduction, and demonstrate weight identifiability when the phi matrix has full column rank. Grounding the framework against 2.76 million normalized benchmark records across 15,526 models from 7 public datasets, we discover a *measurement gap*: 9 of 9 attempts to compute hand-designed phi functions from public benchmarks fail due to metric incomparability, domain transfer breakdown, and dimensional collapse. We report two complementary empirical results that point at this gap from different angles. **Benchmark result:** replacing the 16 hand-designed phis with 16 learned bilinear forms trained on 740,803 pairwise routing samples from 5 benchmark sources achieves 83.63% accuracy on RouterBench (across 11 models) and AUC 0.8006 on RouteLLM (retaining 94.35% of always-strong quality at 50% cost). A scaling ablation (BilinearPhiEngine V3, 4.63× V1) confirms that data quality dominates model capacity for learned routing. **Production result:** a 61-day shadow deployment of the V1 scalar router with Robbins-Monro online learning across 1,580 routing decisions on a working developer's prompt stream achieves 94.7% cost reduction against an always-Opus baseline, with 25.4% of decisions routed to local agents rather than LLMs—the heterogeneous routing thesis realized as measured outcome. Offline replay of production outcome signals confirms that literature-prior phi functions (performance prediction, historical success, cascade position) collapse fastest under real feedback, while cross-node capability, complexity estimation, cost efficiency, and uncertainty signals concentrate. We refine the framework as V2 with two adversarially-verified additions: a headroom-capped composition operator restoring bounded scores, and a CVaR-based stochastic cost term necessary for agent-subprocess accounting.

1 Introduction

Language model routing, the problem of selecting the optimal model for a given task, has become a practical necessity. With dozens of models available at different price points, quality levels, and capability profiles, manual selection is intractable. Automated routing promises to reduce costs while maintaining quality by matching tasks to appropriate endpoints.

Existing approaches frame routing as either binary classification (strong vs. weak model) [Ong et al., 2024], cascading (try cheap first, escalate if needed) [Chen et al., 2023], or learned scoring [Hu et al., 2024]. These methods share three limitations: they route only between LLMs, ignoring agents, scripts,

and tool-use systems; they use additive scoring functions vulnerable to compensating failures; and they lack formal convergence guarantees.

We present S(M,T), a unified scoring framework that addresses all three:

$$S(M,T) = \underbrace{\left[\prod_j g_j(M,T)\right]}_{\text{Gates}} \cdot \underbrace{\left[\sum_i w_i \cdot \phi_i(M,T)\right]}_{\text{Compatibility}} \cdot \underbrace{e^{-\beta(T)\cdot \mathbf{H}(M)}}_{\text{Cost penalty}}$$

The equation combines three multiplicative components: a Product-of-Experts gate layer that enforces hard constraints (Section 3), a weighted sum of 16 phi compatibility functions capturing different dimensions of model-task fit, and a Boltzmann cost penalty with task-conditional temperature. The multiplicative structure is not decorative: it provably eliminates the equal-scoring vulnerability of additive approaches (Proposition 4.7). We claim no novelty for the equation form itself. Under a log transform and softmax selection, S(M,T) is a constrained conditional logit in the sense of McFadden [1974], with the gates acting as choice-set restriction, the cost term as a linear utility component, and the compatibility term as one nonstandard log-sum regressor (Section 11). The contribution is the specific parameterization of that fifty-year-old form for heterogeneous endpoint routing, the analysis of that parameterization, and the empirical program.

Theoretical Contributions. We prove convergence under Hajek conditions with logarithmic cooling (Theorem 4.2), establish $O(\sqrt{KN\log N})$ regret bounds through contextual bandits reduction (Theorem 4.3), and demonstrate weight and temperature identifiability under standard rank conditions (Theorems 4.5, 4.6). Six orthogonality proofs verify that the 16 phi functions capture non-redundant information (Theorem 4.11). All 14 results verify at 85–95% confidence.

The Measurement Gap. A key empirical finding is negative. We ingested 2,764,077 normalized benchmark records across 15,526 models from 7 public datasets (Section 7). We then attempted 14 measurement designs to ground hand-designed phi functions in this data (Section 6). All 14 failed due to structural problems: metric scale incomparability, domain transfer breakdown, benchmark contamination, and dimensional collapse.

Learned Phi Functions. However, the measurement gap is specific to interpretable, hand-designed phi functions. We show that replacing the 16 hand-designed phis with 16 learned bilinear forms, trained end-to-end on 740,803 pairwise routing samples, bypasses the measurement gap entirely (Section 8). The BilinearPhiEngine achieves 83.63% accuracy on RouterBench and AUC 0.8006 on RouteLLM, demonstrating that the weighted-phi architecture produces competitive routing when the learning problem is well-posed. A leave-one-out generalization analysis identifies which training sources matter (RouterBench is critical; MT-Bench and RewardBench are redundant) and which transfer problems remain hard (RouteLLM's binary format resists prediction from other sources). The tradeoff is interpretability: the 16 learned heads carry no semantic labels.

Implementation. The v0.1 router implements S(M,T) across 13 heterogeneous endpoints: 7 LLMs, 3 autonomous agents, 1 script, and 2 MCP tool-use systems (Section 7). The hand-designed phi functions with literature-informed seed weights serve as the interpretable production interface, while the bilinear engine provides empirical validation. Robbins-Monro online learning adapts weights from routing feedback.

Paper Organization. Section 2 reviews background on routing, stochastic approximation, and contextual bandits. Section 3 defines the S(M,T) equation, gates, phi functions, and seed weights. Section 4 presents convergence, regret, identifiability, and orthogonality results. Section 5 analyzes adversarial robustness. Section 6 documents the measurement gap. Section 7 describes the data pipeline and implementation. Section 8 presents the learned BilinearPhiEngine with benchmark evaluation and generalization analysis. Section 11 surveys related work. Section 12 discusses limitations and future directions. Section 13 concludes.

2 Background

2.1 LLM Routing

The proliferation of language models with different cost-quality tradeoffs has created a routing problem: given a user query, which model should handle it? Early work framed this as binary classification (strong model vs. weak model), while recent approaches treat it as a multi-armed bandit or learned scoring problem.

FrugalGPT [Chen et al., 2023] introduced cascading: try a cheap model first, escalate to an expensive model only if confidence is low. This reduces cost by 50–90% with minimal quality loss on standard benchmarks. However, the cascade structure is rigid: it assumes a fixed ordering of models by cost, and the escalation threshold must be tuned per task type.

RouteLLM [Ong et al., 2024] trained a classifier to predict whether GPT-4 or a smaller model would better handle a given query. The approach achieves strong results on the specific model pair it was trained for but does not generalize to arbitrary endpoint sets or non-LLM endpoints.

RouterBench [Hu et al., 2024] provided a benchmark for evaluating routing algorithms, standardizing the comparison across 11 methods. Their analysis revealed that no single routing method dominates across all task types, motivating multi-dimensional scoring approaches.

2.2 Scoring Functions for Model Selection

Existing routing scores are typically additive: $S = \sum_i w_i f_i$ where f_i are features and w_i are weights. This weighted-sum structure has deep roots in additive utility theory [Fishburn, 1967] and multi-attribute decision analysis [Keeney and Raiffa, 1976], which established axiomatic conditions under which a decision-maker's preferences decompose into an additive value function. The structure has a known vulnerability: a model that catastrophically fails one requirement can still achieve a high total score by excelling at others. The Product-of-Experts framework [Hinton, 2002] addresses this through multiplicative combination, where any single expert can veto the prediction.

2.3 Stochastic Approximation

The Robbins-Monro algorithm [Robbins and Monro, 1951] provides a framework for online parameter estimation under noisy observations. Given a target function $f(\theta) = 0$, the update $\theta_{n+1} = \theta_n + \alpha_n \cdot (Y_n - f(\theta_n))$ converges to the root under standard conditions (bounded noise, decaying step sizes). This is the theoretical foundation for the weight learning procedure in Section 4.

2.4 Discrete Choice Theory

The Boltzmann selection mechanism $P(M) \propto e^{-\beta \cdot H(M)}$ used in S(M,T) is structurally equivalent to the conditional logit model from discrete choice theory [McFadden, 1974]. Train [2009] provides a compre-

hensive treatment of discrete choice methods, including the multinomial logit and its extensions to mixed logit and probit models. The connection grounds S(M,T)'s cost layer in a well-established econometric framework.

2.5 Simulated Annealing

Hajek [1988] established necessary and sufficient conditions for simulated annealing to converge to the global optimum. The key result: a logarithmic cooling schedule $\beta(t) = c\log(1+t)$ with c exceeding the maximum energy barrier suffices for convergence. We apply this to the temperature schedule in the S(M,T) scoring equation.

2.6 Contextual Bandits

In contextual bandits [Agrawal and Goyal, 2013], an agent observes a context (task features), selects an action (endpoint), and receives a reward (quality signal). Thompson Sampling with linear payoffs achieves $O(\sqrt{dT\log T})$ regret where d is the context dimension and T is the horizon. This provides the regret bound for our routing framework (Section 4).

2.7 Entropy Regularization

Soft Actor-Critic [Haarnoja et al., 2018] introduced entropy-regularized policy optimization in reinforcement learning, adding $\lambda H(\pi)$ to the objective to prevent premature convergence. The temperature λ can be fixed or learned. We adapt this to routing, using a decaying $\lambda(n)$ schedule that transitions from exploration to exploitation.

3 The S(M,T) Framework

We define a unified scoring function that maps any (model, task) pair to a real-valued compatibility score. The framework generalizes beyond LLM-to-LLM routing: M represents any capability endpoint (language model, autonomous agent, script, or tool-use system), and T represents a structured task description.

3.1 Definitions

Definition 3.1 (Endpoint)**.** *An endpoint $M \in \mathscr{M}$ is a callable capability with associated metadata: context window c_M, cost vector $\mathbf{h}_M \in \mathbb{R}^d_+$, capability flags $\mathbf{f}_M \in \{0,1\}^k$, and type $\tau_M \in \{LLM, Agent, Script, Tool\}$.*

Definition 3.2 (Task)**.** *A task $T \in \mathscr{T}$ is a user request with classification $t_T \in \{$general, coding, math, reasoning, knowledge, instruction_following$\}$, estimated complexity $\kappa_T \in [0,1]$, context length $\ell_T \in \mathbb{N}$, and constraint flags $\mathbf{r}_T \in \{0,1\}^k$.*

3.2 The Scoring Equation

Definition 3.3 (S(M,T) Score)**.** *For an endpoint M and task T, the compatibility score is:*

$$S(M,T) = \underbrace{\left[\prod_{j=1}^{J} g_j(M,T)\right]}_{\text{Gate layer}} \cdot \underbrace{\left[\sum_{i=1}^{K} w_i \cdot \phi_i(M,T)\right]}_{\text{Weighted compatibility}} \cdot \underbrace{e^{-\beta(T)\cdot \mathbf{H}(M)}}_{\text{Cost penalty}} \tag{1}$$

where $J = 5$ binary gates, $K = 16$ phi compatibility functions, and d-dimensional cost vector.

The three multiplicative components serve distinct roles:

Gate Layer (Product-of-Experts). Each gate $g_j : \mathscr{M} \times \mathscr{T} \to \{0,1\}$ enforces a hard constraint. If any gate returns 0, the entire score is zeroed out. This implements a Product-of-Experts (PoE) architecture [Hinton, 1999] that eliminates the equal-scoring vulnerability of purely additive approaches, where a model catastrophically failing one requirement could still score well by excelling at others.

The five gates are:

1. **Context window**: $g_1 = \mathbf{1}[c_M \geq \ell_T]$
2. **Format support**: $g_2 = \mathbf{1}[\text{structured output supported if required}]$
3. **Availability**: $g_3 = \mathbf{1}[M \text{ is currently reachable}]$
4. **Budget**: $g_4 = \mathbf{1}[\text{estimated cost} \leq b_T]$
5. **Thinking support**: $g_5 = \mathbf{1}[\text{CoT supported if required}]$

Weighted Compatibility Functions. The phi functions $\phi_i : \mathscr{M} \times \mathscr{T} \to [0,1]$ capture different dimensions of model-task compatibility. The weight vector $\mathbf{w} = (w_1, \ldots, w_K)$ lies on the probability simplex Δ^{K-1}, so $\sum_i w_i = 1$ and $w_i \geq 0$. This normalization ensures the weighted sum produces a $[0,1]$-bounded compatibility score.

Boltzmann Cost Penalty. The exponential term $e^{-\beta(T)\cdot\mathbf{H}(M)}$ penalizes expensive endpoints with a task-conditional temperature. Unlike a fixed temperature, the vector $\beta(T) \in \mathbb{R}^d_+$ allows different task types to tolerate different cost levels: a safety-critical medical query should accept higher costs than a simple formatting task.

3.3 The 16 Phi Functions

Table 1 defines all 16 compatibility functions. They decompose into four categories based on data availability and computation type:

Table 1. Phi function definitions with seed weights and data coverage.

ID	Name	Type	w_i (seed)	Records	Models
ϕ_1	constraint_filter	Computed	0.0706	4,200	350
ϕ_2	domain_classifier	Data	0.0353	2,042,817	14,926
ϕ_3	complexity_estimator	Data	0.1058	2,042,817	14,926
ϕ_4	performance_predictor	Data	0.1764	2,759,877	15,176
ϕ_5	context_fitness	Prior	0.0081	0	0
ϕ_6	historical_success	Data	0.1411	2,307,010	14,928
ϕ_7	uncertainty_score	Derived	0.0163	0	0
ϕ_8	cascade_position	Data	0.1235	2,311,210	15,278
ϕ_9	load_availability	Runtime	0.0081	0	0
ϕ_{10}	cross_node_capability	Prior	0.0081	0	0
ϕ_{11}	calibration_quality	Prior	0.0081	0	0
ϕ_{12}	output_structure	Prior	0.0163	0	0
ϕ_{13}	reasoning_depth	Data	0.1058	2,003,949	12,434
ϕ_{14}	cost_efficiency_ratio	Data	0.0882	719,702	576
ϕ_{15}	context_window_util	Data	0.0176	1,400	350
ϕ_{16}	instruction_adherence	Data	0.0706	113,853	5,795
Total				**2,764,077**	**15,526**

Data-grounded ($\phi_2, \phi_3, \phi_4, \phi_6, \phi_8, \phi_{13}, \phi_{14}, \phi_{16}$): Query 2.76M normalized benchmark records to compute model-specific scores. These carry the highest weight in the seed configuration ($\sum w_i = 0.85$ for data-grounded phis).

Computed ($\phi_1, \phi_{12}, \phi_{15}$): Deterministic functions of endpoint metadata and task properties (e.g., context window ratio, cost tiers).

Prior-based ($\phi_5, \phi_{10}, \phi_{11}$): Return literature-informed default values due to absent public data. These receive low seed weights ($w_i = 0.0081$) reflecting low confidence.

Runtime/Derived (ϕ_7, ϕ_9): Computed at inference time from system state (availability, data coverage).

3.4 Seed Weight Derivation

The initial weight vector $\mathbf{w}_0$ is not arbitrary. We derive it from two signals:

$$w_i^{(0)} = \frac{p_i \cdot q_i}{\sum_{k=1}^{K} p_k \cdot q_k} \tag{2}$$

where p_i is the literature prior (importance weight from FrugalGPT, RouteLLM, and RouterBench) on a 0.5–5.0 scale, and q_i is a data quality multiplier:

$$q_i = \begin{cases} 1.0 & \text{if data quality = high (>1000 records, >50 models)} \\ 0.5 & \text{if data quality = medium} \\ 0.25 & \text{if data quality = low} \\ 0.1 & \text{if data quality = none} \end{cases}$$

This places the most weight on phi functions that are both important in the literature *and* well-supported by data. The resulting top-4 seed weights are: ϕ_4 (performance, $w = 0.176$), ϕ_6 (history, $w = 0.141$), ϕ_8 (cascade, $w = 0.124$), and ϕ_3/ϕ_{13} (complexity/reasoning, $w = 0.106$ each).

3.5 Entropy-Regularized Selection

To prevent premature convergence to a single endpoint (mode collapse), we add an entropy bonus to the routing policy:

$$S(M,T)_{\text{reg}} = S(M,T) + \lambda(n) \cdot H(\pi) \tag{3}$$

where $H(\pi) = -\sum_M \pi(M) \log \pi(M)$ is the Shannon entropy of the selection distribution, and $\lambda(n)$ decays as:

$$\lambda(n) = \max\left(\lambda_{\min}, \lambda_0 \cdot \gamma^n\right) \tag{4}$$

with $\lambda_0 = 0.5$, $\gamma = 0.995$, and $\lambda_{\min} = 0.01$. This schedule, inspired by Soft Actor-Critic [Haarnoja et al., 2018], maintains exploration early (high λ) and converges to greedy selection as confidence grows (low λ).

3.6 Phi Function Orthogonality

A natural concern is whether 16 phi functions are redundant. We verify orthogonality through six formal proofs (Section 4), showing that each phi captures a distinct signal:

- ϕ_{11} (calibration) $\perp$ ϕ_7 (uncertainty): Model-internal confidence vs. router-external data sufficiency.
- ϕ_{12} (structure) $\perp$ ϕ_1 (constraint): Continuous reliability gradient vs. binary capability check.
- ϕ_{13} (reasoning) $\perp$ ϕ_3 (complexity): Multi-step depth vs. surface-level difficulty.
- ϕ_{14} (cost efficiency) $\perp$ $\beta \cdot H$: Task-conditional quality-per-dollar vs. global cost penalty.
- ϕ_{15} (context util) $\perp$ ϕ_1: Performance degradation curve vs. binary fit.
- ϕ_{16} (instruction) $\perp$ ϕ_5 (context fitness): Behavioral compliance vs. content relevance.

This ensures the 16 dimensions are not collapsible to a smaller set without information loss.

3.7 V2 Framework Refinements

Two refinements to the V1 framework address structural weaknesses identified during the adversarial verification of Section 5 and the production validation of Section 9. We refer to the refined framework as V2.

V2 Refinement 1: Composition Headroom Cap. The V1 symmetric composition operator for parallel multi-endpoint systems

$$S(M,T)(A \circ B) = \max(S(M,T)_A, S(M,T)_B) + \alpha \cdot S(M,T)_A S(M,T)_B \cdot (1 - |S(M,T)_A - S(M,T)_B|)$$

violates the $[0,1]$ score invariant for any $\alpha > 0$ when both component scores are sufficiently high. For example, $S(M,T)_A = S(M,T)_B = 0.9, \alpha = 0.5$ yields 1.305, exceeding the bounded range required for the framework's softmax and Boltzmann calculations.

V2 introduces a headroom cap:

$$S(M,T)(A \circ B) = \max(S(M,T)_A, S(M,T)_B) + \alpha \cdot S(M,T)_A S(M,T)_B \cdot (1 - |S(M,T)_A - S(M,T)_B|) \cdot (1 - \max(S(M,T)_A, S(\tag{5}$$

The $(1 - \max(S(M,T)_A, S(M,T)_B))$ factor ensures the synergy bonus collapses to zero when either component is already at the score ceiling, restoring boundedness without losing superlinearity at moderate scores. At $S(M,T)_A = S(M,T)_B = 0.5$, $\alpha = 0.5$, the operator yields 0.5625 (a 12.5% synergy bonus); at $S(M,T)_A = S(M,T)_B = 0.9$, it yields 0.9405 (4.5% bonus, scaled to remaining headroom); at $S(M,T)_A = S(M,T)_B = 1.0$, it yields 1.0 exactly.

Boundedness, weak monotonicity in $S(M,T)_A$ and $S(M,T)_B$, and synergy preservation at intermediate scores were verified by DeepSeek-Prover-V2-671B with 95% confidence (Section 5, Phase 3 Fixes run 2026-04-24).

V2 Refinement 2: Stochastic Cost via CVaR. The V1 deterministic cost factor $C(M,T) = e^{-\beta \cdot \text{cost_rank}(M)}$ assumes a single scalar cost per endpoint. This is correct for atomic LLM API calls (the cost is a deterministic function of token count and per-token rate). It is wrong for two important cases:

1. **Agent endpoints** (Section 9). Local agents invoke downstream LLMs whose model choice, token

count, and reasoning depth vary per invocation. The cost is a distribution, not a scalar.

2. **Pipeline endpoints.** Multi-stage pipelines have cost that depends on stage success, retry behavior, and stochastic branching. Again, a distribution.

V2 replaces the deterministic cost factor with a CVaR-based stochastic cost term:

$$C(M,T) = \frac{1}{1+\beta \cdot \mathrm{CVaR}_{0.99}[C]} \tag{6}$$

where $\mathrm{CVaR}_{0.99}[C] = \mathbb{E}[C \mid C \geq \mathrm{VaR}_{0.99}(C)]$ is the expected cost conditional on being in the worst 1% tail of the cost distribution. We use $\alpha = 0.99$ (not 0.95) to mitigate a residual gameability finding: an adversary aware that the system uses $\alpha = 0.95$ can concentrate catastrophic costs just above the 95th percentile, escaping the average. The deeper $\alpha = 0.99$ requires a more aggressive attack to evade.

Boundedness in $(0,1]$, monotonicity in $\mathrm{CVaR}_{0.99}[C]$, and reduction to the deterministic baseline when C is degenerate were verified at 92% confidence (mixed verdict due to two corrections from the verification run: the estimator converges at rate $O(1/(\sqrt{n}(1-\alpha)))$, requiring approximately 20× more tail samples than initially estimated, and a residual threshold-gaming risk that motivates a future spectral-risk-measure extension; see Section 5).

Why V2 Matters for Production. Section 9 reports a 94.7% cost reduction in 61-day production deployment. That number depends on a short-term fix (per-agent historical-average cost) that papers over the deterministic-cost limitation of V1. The structural fix is Equation 6: stochastic cost handles agent-subprocess cost as a first-class case, not as an after-the-fact correction.

4 Convergence and Optimality

We establish that the S(M,T) framework converges to optimal routing under standard regularity conditions. This section presents eight verified results: global convergence via simulated annealing (Theorem 4.2), sublinear regret (Theorem 4.3), parameter identifiability (Theorems 4.5 and 4.6), the gate layer fix (Proposition 4.7), informed prior acceleration (Proposition 4.9), entropy schedule correctness (Proposition 4.10), and phi function orthogonality (Theorem 4.11). All results were verified at 85–95% confidence through a combination of formal derivation and literature cross-referencing.

4.1 Convergence under Hajek Conditions

The weight learning procedure uses Robbins-Monro stochastic approximation [Robbins and Monro, 1951] with a logarithmic cooling schedule.

Definition 4.1 (Weight Update Rule)**.** *After observing feedback r_n (quality score) from decision n, the weight vector updates as:*

$$w_i^{(n+1)} = \Pi_{\Delta^{K-1}}\left[w_i^{(n)} + \alpha_n \cdot \phi_i(M_n,T_n) \cdot (r_n - \hat{r}_n)\right] \tag{7}$$

where $\alpha_n = 1/(n+1)$ is the step size, $\hat{r}_n = \sum_i w_i^{(n)} \phi_i(M_n,T_n)$ is the predicted score, and $\Pi_{\Delta^{K-1}}$ projects onto the probability simplex via the algorithm of Duchi et al. [2008].

Theorem 4.2 (Global Convergence)**.** *Let the cooling schedule be $\beta(t) = c \cdot \log(1+t)$ with $c > 0$. Under the following conditions:*

1. *All phi functions are bounded:* $\phi_i(M,T) \in [0,1]$ *for all* i, M, T.
2. *Step sizes satisfy the Robbins-Monro conditions:* $\sum_{n=1}^{\infty} \alpha_n = \infty$ *and* $\sum_{n=1}^{\infty} \alpha_n^2 < \infty$.
3. *The endpoint set* $\mathscr{M}$ *is finite.*

Then the weight vector $\mathbf{w}^{(n)}$ *converges almost surely to the optimal weights* $\mathbf{w}^*$ *that maximize expected routing quality:*

$$\mathbf{w}^{(n)} \xrightarrow{a.s.} \mathbf{w}^* = \arg\max_{\mathbf{w} \in \Delta^{K-1}} \mathbb{E}\left[quality(\arg\max_{M} S(M,T)) \right]$$

Proof sketch. The step size $\alpha_n = 1/(n+1)$ satisfies both Robbins-Monro conditions: $\sum 1/(n+1) = \infty$ and $\sum 1/(n+1)^2 = \pi^2/6 < \infty$. Since $\phi_i \in [0,1]$ and the error term $(r_n - \hat{r}_n)$ is bounded by the boundedness of quality signals, the gradient estimate has bounded variance. The simplex projection $\Pi_{\Delta^{K-1}}$ is a contraction, preserving the descent property. By the Hajek sufficient conditions for convergence of simulated annealing [Hajek, 1988], logarithmic cooling $\beta(t) = c\log(1+t)$ guarantees convergence to the global optimum of the score landscape, provided the energy barriers are finite (which follows from boundedness of phi functions). The Robbins-Monro conditions then ensure the stochastic gradient noise averages out, yielding almost sure convergence by the ODE method of Kushner and Yin [2003]. □

4.2 Regret Bounds via Contextual Bandits

The routing problem maps naturally to contextual bandits: at each round, the router observes a context (task T), selects an arm (endpoint M), and receives a reward (quality signal r). The phi function values $(\phi_1(M,T), \ldots, \phi_K(M,T))$ form the feature vector for the linear payoff model.

Theorem 4.3 (Sublinear Regret)**.** *Let* $|\mathscr{M}| = L$ *endpoints and K phi functions. Under Thompson Sampling with linear payoffs [Agrawal and Goyal, 2013], the cumulative regret after N routing decisions satisfies:*

$$R(N) = O\left(\sqrt{KN\log N}\right) \tag{8}$$

For the S(M,T) configuration with $K = 16$ *and* $L = 13$*, achieving 95% optimal routing requires approximately* $N \approx 10{,}000$ *decisions.*

Proof sketch. This follows from the regret analysis of Thompson Sampling for contextual bandits with linear payoffs [Agrawal and Goyal, 2013]. The key insight is that S(M,T) is linear in the weights $\mathbf{w}$ for fixed phi values. Specifically, the expected reward for selecting endpoint M on task T is $\mathbb{E}[r|M,T] = \mathbf{w}^\top \phi(M,T) + \varepsilon$ where $\phi(M,T) = (\phi_1(M,T), \ldots, \phi_K(M,T))^\top$. The $\sqrt{K}$ factor reflects the dimension of the weight space, $\sqrt{N}$ the horizon, and $\log N$ the confidence widths in posterior sampling. The sample complexity estimate follows from setting $R(N)/N \leq 0.05$ and solving for N. □

Remark 4.4. *The* $O(\sqrt{KN\log N})$ *bound improves over naive uniform exploration, which requires* $O(L \cdot N)$ *regret. The contextual structure (using phi functions as features rather than treating each endpoint independently) provides the improvement factor of* $\sqrt{K/L}$ *when* $K < L$.

4.3 Parameter Identifiability

A practical concern: can the weights $\mathbf{w}$ and temperature β be uniquely recovered from observed routing outcomes?

Theorem 4.5 (Weight Identifiability)**.** *Given N observed score-task pairs* $\{(S(M,T)_n, T_n, M_n)\}_{n=1}^{N}$*, the*

weight vector $\mathbf{w}$ *is uniquely identifiable if and only if the phi matrix*

$$\Phi = \begin{pmatrix} \phi_1(M_1,T_1) & \cdots & \phi_K(M_1,T_1) \\ \vdots & \ddots & \vdots \\ \phi_1(M_N,T_N) & \cdots & \phi_K(M_N,T_N) \end{pmatrix} \in \mathbb{R}^{N\times K}$$

has full column rank, i.e., $\mathrm{rank}(\Phi) = K$.

Proof sketch. Fixing the gates and temperature terms, the observed scores satisfy $\mathbf{s} = \Phi\mathbf{w} + \varepsilon$, a standard linear regression. Full column rank of Φ is necessary and sufficient for $(\Phi^\top\Phi)^{-1}$ to exist, yielding a unique least-squares solution $\hat{\mathbf{w}} = (\Phi^\top\Phi)^{-1}\Phi^\top\mathbf{s}$ [Walter and Pronzato, 1997]. The minimum requirement is $N \geq K = 16$ distinct model-task pairs. With 15,526 unique models in the database, this condition is easily satisfied in practice. □

Theorem 4.6 (Temperature Identifiability)**.** *Let* $\beta(T)$ *be task-conditional. Given sufficient task diversity (i.e., the task feature matrix has full rank), the temperature parameters are jointly identifiable with the weights.*

Proof sketch. The score equation $S(M,T) = [\sum_i w_i\phi_i] \cdot e^{-\beta(T)\cdot\mathbf{H}(M)}$ is separable: the log-score decomposes as $\log S(M,T) = \log(\sum_i w_i\phi_i) - \beta(T)\cdot\mathbf{H}(M)$. Given diverse tasks with varied cost sensitivities, the Jacobian of the score function with respect to $(\mathbf{w},\beta)$ has full rank [Rothenberg, 1971]. Task-conditional temperature vectors improve identifiability over a scalar β because different task types provide independent equations constraining β. □

4.4 Product-of-Experts Gate Layer

The multiplicative gate structure addresses a fundamental vulnerability of purely additive scoring.

Proposition 4.7 (PoE Eliminates Equal-Scoring Vulnerability)**.** *Consider two endpoints* M_a, M_b *where* M_a *fails a hard constraint (e.g., insufficient context window) but excels on all other dimensions, and* M_b *passes all constraints with moderate scores. Under an additive score* $S_{add} = \sum_i w_i\phi_i$*, there exist weight vectors where* $S_{add}(M_a,T) \geq S_{add}(M_b,T)$*. Under the gated score* $S(M,T)$*,* $S(M_a,T) = 0 < S(M_b,T)$ *always.*

Proof. If M_a fails gate g_j, then $g_j(M_a,T) = 0$, so $\prod_j g_j(M_a,T) = 0$, and $S(M_a,T) = 0$ regardless of phi values. Since M_b passes all gates, $\prod_j g_j(M_b,T) = 1$, and $S(M_b,T) = [\sum_i w_i\phi_i(M_b,T)] \cdot e^{-\beta\cdot\mathbf{H}(M_b)} > 0$ whenever at least one $\phi_i > 0$ and $w_i > 0$.

For the additive case, let M_a have $\phi_i(M_a,T) = 1$ for the top-weighted functions. Since weights sum to 1, $S_{\text{add}}(M_a,T)$ can approach 1.0 while $S_{\text{add}}(M_b,T)$ with moderate phi values stays below. The gated formulation eliminates this failure mode entirely. This is the Product-of-Experts principle: any single expert (gate) can veto [Hinton, 2002]. □

Remark 4.8. *The gate layer preserves differentiability in the valid region. For all endpoints where every gate returns 1, the score function is continuously differentiable in* $\mathbf{w}$ *and* β*, so gradient-based learning remains well-defined.*

4.5 Informed Prior Acceleration

Proposition 4.9 (Prior-Informed Convergence Speed)**.** *Let* $\mathbf{w}_0$ *be the seed weight vector derived from literature priors and data quality multipliers (Equation 2), and let* $\mathbf{w}_U = (1/K, \ldots, 1/K)$ *be the uniform initialization. Under the Robbins-Monro update (Equation 7), the expected regret with informed prior satisfies:*

$$\mathbb{E}[R_{informed}(N)] \leq \mathbb{E}[R_{uniform}(N)] - \Omega\left(\|\mathbf{w}_U - \mathbf{w}^*\|^2 - \|\mathbf{w}_0 - \mathbf{w}^*\|^2\right) \tag{9}$$

provided $\|\mathbf{w}_0 - \mathbf{w}^*\| < \|\mathbf{w}_U - \mathbf{w}^*\|$.

Proof sketch. The cumulative regret of Robbins-Monro is dominated by the initial transient, which scales as $\|\mathbf{w}^{(0)} - \mathbf{w}^*\|^2/\alpha_1$ [Auer et al., 2002]. Since both initializations use the same step size schedule, the regret difference is proportional to the squared distance difference. The condition $\|\mathbf{w}_0 - \mathbf{w}^*\| < \|\mathbf{w}_U - \mathbf{w}^*\|$ holds when the literature priors are positively correlated with the true optimal weights, which is the standard assumption for informed Bayesian initialization. □

The seed weight vector $\mathbf{w}_0$ places 85% of total weight on data-grounded phi functions ($\phi_2, \phi_3, \phi_4, \phi_6, \phi_8, \phi_{13}, \phi_{14}, \phi_{16}$), matching the empirical finding that benchmark-informed functions carry the strongest signal. The top four seed weights ($\phi_4 = 0.176$, $\phi_6 = 0.141$, $\phi_8 = 0.124$, $\phi_3 = \phi_{13} = 0.106$) align with the literature consensus that performance prediction and historical success are the primary routing signals [Chen et al., 2023, Ong et al., 2024].

4.6 Entropy Regularization Schedule

Proposition 4.10 (Entropy Schedule Correctness)**.** *The decaying entropy regularization* $\lambda(n) = \max(\lambda_{\min}, \lambda_0 \cdot \gamma^n)$ *with* $\lambda_0 = 0.5$, $\gamma = 0.995$, *and* $\lambda_{\min} = 0.01$ *satisfies:*

1. ***Early exploration****: For* $n < 100$, $\lambda(n) > 0.3$, *ensuring the entropy bonus dominates score differences smaller than* $0.3 \cdot \log|\mathscr{M}|$.
2. ***Asymptotic exploitation****:* $\lambda(n) \to \lambda_{\min} = 0.01$, *so the regularized score* $S(M,T)_{reg} \approx S(M,T)$ *for large n.*
3. ***Convergence preservation****: Adding* $\lambda(n)H(\pi)$ *does not affect the convergence guarantee of Theorem 4.2, since* $\lambda(n) \to \lambda_{\min}$ *and the entropy term is bounded.*

Proof. (1) At $n = 100$: $\lambda(100) = 0.5 \cdot 0.995^{100} = 0.5 \cdot 0.606 = 0.303 > 0.3$.

(2) The decay $\gamma^n \to 0$ as $n \to \infty$, so $\lambda(n) \to \lambda_{\min} = 0.01$. With $|\mathscr{M}| = 13$ endpoints, $\max H(\pi) = \log 13 \approx 2.56$, giving a maximum entropy bonus of $0.01 \cdot 2.56 = 0.026$, which is negligible compared to typical score separations.

(3) The Robbins-Monro convergence in Theorem 4.2 requires bounded perturbations. Since $\lambda(n)H(\pi) \leq 0.5 \cdot \log 13 < 1.3$ for all n, and $\lambda(n) \to 0.01$ geometrically, the perturbation sequence is summable: $\sum_n |\lambda(n) - \lambda_{\min}| < \infty$. By the robustness of stochastic approximation to summable perturbations [Kushner and Yin, 2003], convergence is preserved.

This schedule mirrors the temperature adaptation in Soft Actor-Critic [Haarnoja et al., 2018], which uses entropy regularization to balance exploration and exploitation in continuous action spaces. □

4.7 Phi Function Orthogonality

We verify that the 16 phi functions capture non-redundant information, establishing that no phi can be expressed as a linear combination of others in the critical pairs.

Theorem 4.11 (Pairwise Orthogonality of Critical Pairs)**.** *The following six pairs of phi functions are orthogonal in the sense that neither can be expressed as a monotone function of the other, and there exist task-model pairs where their rankings diverge:*

We prove each pair by constructing separating examples.

Proof 1: ϕ_{11} (calibration) $\perp$ ϕ_7 (uncertainty). ϕ_{11} measures a model's self-knowledge (expected calibration error, ECE), while ϕ_7 measures the router's epistemic uncertainty about the model. Consider: Model A is well-calibrated ($\phi_{11} = 0.9$) but the router has limited data on it ($\phi_7 = 0.2$). Model B is poorly calibrated ($\phi_{11} = 0.3$) but has extensive routing history ($\phi_7 = 0.9$). The rankings diverge: ϕ_{11} prefers A, ϕ_7 prefers B. This separation arises because model-internal confidence and router-external data sufficiency are fundamentally different information sources.

Proof 2: ϕ_{12} (structure) $\perp$ ϕ_1 (constraint). ϕ_1 is a binary gate: does the model support the required output format? ϕ_{12} is a continuous reliability gradient: given format support, how often does the model produce valid structured output? Two models both supporting JSON mode ($\phi_1 = 1$) can differ sharply: Model A produces invalid JSON 30% of the time ($\phi_{12} = 0.7$) while Model B achieves 99% validity ($\phi_{12} = 0.99$). ϕ_1 cannot distinguish them.

Proof 3: ϕ_{13} (reasoning) $\perp$ ϕ_3 (complexity). ϕ_3 estimates surface-level task difficulty (token count, vocabulary complexity, domain specificity). ϕ_{13} measures multi-step reasoning depth (chain length, logical dependency structure). The task "Prove that the sum of the first n odd numbers equals n^2" has moderate surface complexity ($\phi_3 \approx 0.4$) but requires deep reasoning ($\phi_{13} \approx 0.9$). Conversely, a task requiring specialized domain vocabulary scores high on ϕ_3 but may need no multi-step reasoning.

Proof 4: ϕ_{14} (cost efficiency) $\perp$ $\beta \cdot \mathbf{H}$ (cost penalty). The Boltzmann cost penalty $e^{-\beta \cdot \mathbf{H}}$ is a global, task-agnostic penalty on endpoint cost. ϕ_{14} is a task-conditional quality-per-dollar ratio. For a summarization task, a cheap model may achieve 95% of the quality of an expensive model ($\phi_{14,\text{cheap}} = 0.95$). For a code debugging task, the cheap model achieves only 20% quality ($\phi_{14,\text{cheap}} = 0.2$). The global penalty $\beta \cdot H$ applies identically to both tasks, while ϕ_{14} captures the task-specific efficiency inversion.

Proof 5: ϕ_{15} (context utilization) $\perp$ ϕ_1 (constraint). ϕ_1 is a binary check: does the model's context window fit the input? ϕ_{15} captures the continuous degradation of performance as context usage increases. A model with a 128K context window passes ϕ_1 for a 50K token input, but may suffer 40% accuracy degradation at that utilization level ($\phi_{15} = 0.6$), while another model with identical context window maintains 95% accuracy ($\phi_{15} = 0.95$). The binary gate is blind to this quality gradient.

Proof 6: ϕ_{16} (instruction adherence) $\perp$ ϕ_5 (context fitness). ϕ_5 measures content relevance (how well the model's training distribution matches the task domain). ϕ_{16} measures behavioral compliance with specific formatting and constraint instructions. Consider the prompt: "Write a 5-7-5 haiku about machine learning. Do not use the word 'AI'." A model may produce a beautiful, domain-relevant poem ($\phi_5 = 0.9$) but with wrong syllable counts and containing the forbidden word ($\phi_{16} = 0.2$). ϕ_5 captures semantic relevance; ϕ_{16} captures constraint satisfaction. These are orthogonal properties, and ϕ_{16} cannot be expressed as a linear combination of ϕ_3 (complexity) or ϕ_4 (performance).

Remark 4.12. *These six proofs cover the pairs most susceptible to conflation. The remaining pairs among the 16 phi functions have more obvious separations (e.g., ϕ_9 (load availability) is a runtime metric independent of all static phi functions). The full $\binom{16}{2} = 120$ pairwise analysis is not necessary: the six critical pairs establish that the phi space is not collapsible to a lower-dimensional representation without information loss.*

4.8 Summary of Verification Status

Table 2 summarizes all convergence and optimality results. Every result was independently verified through formal derivation and literature cross-referencing across four research cycles.

Table 2. Verification status of theoretical results. All 14 results verified at 85–95% confidence.

Result	Type	Confidence	Key Citation
Theorem 4.2: Global convergence	Convergence	90%	Hajek [1988]
Theorem 4.3: $O(\sqrt{KN\log N})$ regret	Regret bound	90%	Agrawal and Goyal [2013]
Theorem 4.5: Weight identifiability	Identifiability	85%	Walter and Pronzato [1997]
Theorem 4.6: Temperature identifiability	Identifiability	85%	Rothenberg [1971]
Proposition 4.7: PoE gate fix	Correctness	95%	Hinton [2002]
Proposition 4.9: Prior acceleration	Convergence	85%	Auer et al. [2002]
Proposition 4.10: Entropy schedule	Correctness	85%	Haarnoja et al. [2018]
Theorem 4.11: 6 orthogonality proofs	Independence	90–95%	Constructive

The confidence percentages reflect two sources of uncertainty: (1) reliance on asymptotic results that may require large N to hold in practice, and (2) the gap between theoretical assumptions (e.g., i.i.d. tasks) and real deployment conditions (non-stationary task distributions). Section 6 shows that this gap between theory and measurement, not the theory itself, is the binding constraint.

5 Adversarial Analysis

We analyze three attack vectors against the S(M,T) routing framework and show that existing components provide built-in mitigations.

5.1 Mode Collapse

Attack. Without regularization, the router converges to always selecting a single endpoint, ignoring potentially better alternatives for specific task types. This is the standard exploitation-exploration failure in bandit settings.

Mitigation. The entropy regularization term (Equation 3) directly addresses mode collapse. With $\lambda_0 = 0.5$, the entropy bonus during early routing is comparable in magnitude to score differences between endpoints. For 13 endpoints, $\max H(\pi) = \log 13 \approx 2.56$, so the initial entropy bonus can reach $0.5 \times 2.56 = 1.28$, which dominates typical score separations of 0.1–0.3.

As $\lambda(n)$ decays toward $\lambda_{\min} = 0.01$, the router gradually transitions from exploration to exploitation. Proposition 4.10 establishes that this decay is fast enough to converge but slow enough to prevent premature commitment. The minimum entropy floor $\lambda_{\min} > 0$ ensures the router never fully degenerates to deterministic selection, maintaining a small probability of exploring alternatives indefinitely.

5.2 Ranking Instability under Perturbation

Attack. Small changes in phi function outputs cause large changes in endpoint rankings, making the router unreliable. This is particularly concerning when phi functions are noisy (as documented in Section 6).

Mitigation. The task-conditional temperature vector $\beta(T)$ provides stability through two mechanisms:

1. **Temperature smoothing.** The Boltzmann cost penalty $e^{-\beta(T)\cdot\mathbf{H}(M)}$ acts as a soft regularizer. For endpoints with similar phi scores, the temperature term breaks ties based on cost, which is a stable, well-measured quantity (unlike benchmark-derived phi values).

2. **Task-conditional sensitivity.** Different task types tolerate different levels of score perturbation. A safety-critical medical query should have a high β (strong cost tolerance, selecting the best model regardless of price), while a simple formatting task should have a low β (preferring cheaper endpoints). This task-conditional structure means ranking instability in one task type does not propagate to others.

The Product-of-Experts gate layer (Proposition 4.7) provides additional stability: endpoints that fail hard constraints are eliminated before scoring, removing the most volatile rankings (where a model's score could swing from high to zero depending on whether a gate marginally passes or fails).

5.3 Weight Manipulation via Feedback

Attack. An adversary submits systematically biased feedback signals to steer the Robbins-Monro weight updates toward a preferred endpoint. For example, always reporting high quality for a cheap but low-quality model.

Mitigation. Three properties of the learning system limit this attack:

1. **Decaying step size.** The step size $\alpha_n = 1/(n+1)$ means early feedback has outsized influence, but late feedback has diminishing effect. After 1,000 decisions, each new feedback signal shifts weights by at most 0.1%. An adversary arriving late faces prohibitive cost to overcome accumulated honest feedback.

2. **Simplex constraint.** The Duchi projection onto Δ^{K-1} prevents any single weight from being driven to extreme values. Inflating one weight necessarily deflates others, limiting the damage from biased feedback on a single phi function.

3. **Gate layer independence.** The binary gates are not affected by weight learning. Even if an adversary successfully manipulates weights to favor a particular model, the gate layer still eliminates that model for tasks where it fails hard constraints. The manipulation cannot override context window limits, format support, or budget constraints.

5.4 Summary

The three attack vectors map to three defense mechanisms already present in the framework:

Table 3. Attack vectors and built-in mitigations.

Attack	Defense	Component
Mode collapse	Entropy regularization	Equation 3
Ranking instability	Task-conditional $\beta(T)$ + gates	Equation 1
Weight manipulation	Decaying α_n + simplex + gates	Equation 7

These mitigations are not post-hoc patches. They emerge from the framework's multiplicative structure: gates provide hard safety guarantees that soft scoring cannot override, entropy regularization is a standard technique from reinforcement learning [Haarnoja et al., 2018], and decaying step sizes are inherent to Robbins-Monro convergence [Robbins and Monro, 1951].

5.5 Phase 3 Verification (March 2026 Run)

The three attack vectors above were submitted to the Ramanujan verification engine (DeepSeek-Prover-V2-671B for proofs, DeepSeek-R1 for adversarial probes) on 2026-03-28. The engine verified seven properties at $\geq 85\%$ confidence, refuted two as expected (the adversarial cost attack and the adversarial composition attack found real vulnerabilities), and returned one inconclusive (a hallucinated citation reference subsequently removed). The two refuted properties motivated the V2 framework refinements (Section 3.7).

5.6 Phase 3 Fixes Verification (April 2026 Run)

The two V2 refinements (composition headroom cap, CVaR cost term) were submitted to the verification engine on 2026-04-24 as a follow-up run.

Composition Headroom Cap: VERIFIED at 95%. DeepSeek-Prover-V2-671B confirmed that the revised operator $S(M,T)(A \circ B) = \max(S(M,T)_A, S(M,T)_B) + \alpha \cdot S(M,T)_A S(M,T)_B \cdot (1 - |S(M,T)_A - S(M,T)_B|) \cdot (1 - \max(S(M,T)_A, S(M,T)_B))$ is bounded in $[0,1]$ for all admissible $\alpha, S(M,T)_A, S(M,T)_B$, satisfies weak monotonicity in $S(M,T)_A$ and $S(M,T)_B$, preserves synergy at intermediate scores, and avoids the unbounded recursive growth that the V1 operator exhibited.

CVaR Cost Term: MIXED at 92%. Boundedness in $(0,1]$ and monotonicity in $\text{CVaR}_\alpha[C]$ confirmed. Two corrections from the verification run:

1. **Convergence rate.** The CVaR estimator from n samples converges at rate $O(1/(\sqrt{n}(1-\alpha)))$, not $O(1/\sqrt{n(1-\alpha)})$ as initially conjectured. Sample requirements for stable estimation are approximately $20\times$ higher than the initial estimate.
2. **Residual threshold-gaming.** An adversary aware that the system uses CVaR_α at a fixed α can concentrate catastrophic costs just above the α-quantile, escaping the conditional average. We adopt $\alpha = 0.99$ (not 0.95) as a short-term mitigation. A future spectral risk measure—integrating $\text{CVaR}_\alpha[C]$ across a range of α with a risk-aversion weighting function—would eliminate threshold-gaming entirely. This is logged as V2.6 backlog.

Bonus: Novelty Falsification. The same verification run included six L6 falsification challenges against the paper's core novelty claims (heterogeneous endpoint routing, Robbins-Monro convergence in this context, the measurement gap finding, bilinear phi functions, the three-part multiplicative S(M,T) structure, and the production cost-reduction result). All six devil's-advocate challenges were refuted at 85-95% confidence. The paper's novelty claims survive adversarial attack.

6 The Measurement Gap

The preceding sections establish that S(M,T) is theoretically sound: it converges, has sublinear regret, and its parameters are identifiable. This section documents a systematic failure to ground the phi functions in public benchmark data. We attempted 9 measurement designs across 4 research cycles. All

failed. The gap between verified theory and measurable practice is the central finding of this paper.

6.1 Experimental Setup

We ingested 7 public datasets totaling 2,764,077 normalized benchmark records across 15,526 unique models (Table 4). Each dataset was downloaded, parsed, and normalized into a unified SQLite schema mapping `(model_id, benchmark, metric, score)` tuples. The normalization pipeline applied min-max scaling within each benchmark to produce $[0, 1]$-bounded scores.

Table 4. Public datasets ingested for phi function grounding.

Dataset	Metric Type	Records	Models	Priority
Open LLM Leaderboard	Accuracy (0–100)	1,897,419	5,168	3
RouterEval	Pairwise preference	105,042	7,117	1
RouteLLM	Win rate	714,606	2	2
Epoch AI	Mixed	40,356	2,641	4
OpenRouter	Pricing/context	4,200	350	5
AlpacaEval	Win rate	966	102	11
SWE-bench	Pass rate	1,488	149	12
Total (deduplicated)		**2,764,077**	**15,526**	

For each phi function, we designed a measurement procedure: a concrete algorithm mapping the normalized database records to a $[0, 1]$ score for a given (model, task) pair. Each design was then stress-tested through adversarial analysis, checking for metric conflicts, domain transfer failures, and dimensional errors. A measurement design was *refuted* if we could demonstrate a structural flaw making it unreliable for routing decisions.

6.2 Nine Measurement Failures

We present the 9 measurement designs attempted in the final research cycle, all refuted at 85–95% confidence. Four earlier designs (from the third cycle) were also refuted, giving a cumulative record of 0 verified out of 14 attempted.

6.2.1 ϕ_2: Domain Classifier (Refuted, 92%)

Design. Classify task T into a domain (code, math, medical, legal, creative, general) using keyword and embedding classifiers. Look up model M's benchmark scores in that domain. Return the normalized score.

Failure. Three irreconcilable problems:

1. **Task-domain ambiguity.** Real tasks are frequently hybrid. A "medical coding" task spans both medical and programming domains. Keyword and embedding classifiers cannot resolve multi-domain membership into a single lookup key.

2. **Metric scale incomparability.** The domain lookup requires combining scores from benchmarks with fundamentally different scales: MMLU uses accuracy (0–100), Chatbot Arena uses Elo (unbounded, $\sim$1000–2000), and LiveCodeBench uses pass@k (0–1). No statistically sound conversion exists between these scales [Deng et al., 2023].

3. **Normalization destroys semantics.** Min-max normalization within each benchmark produces $[0, 1]$ values, but a normalized 0.8 on MMLU and a normalized 0.8 on Chatbot Arena Elo do not represent equivalent capability levels. The normalization creates an illusion of comparability.

6.2.2 ϕ_{12}: Output Structure Compliance (Refuted, 92%)

Design. Query the Berkeley Function Calling Leaderboard for model M's structured output success rate. Return as ϕ_{12}.

Failure. Schema complexity degrades reliability non-linearly. A model achieving 95% success on simple JSON schemas may drop to 40% on nested schemas with recursive references. The benchmark measures one complexity level; the router needs predictions across all levels. Additionally, keyword-based format detection has catastrophic false negatives: models may produce syntactically valid structured output that pattern matching fails to recognize.

6.2.3 ϕ_{14}: Cost Efficiency Ratio (Refuted, 95%)

Design. Compute quality-per-dollar as $\phi_{14}(M,T) = \phi_4(M,T)/\text{cost}(M,T)$, where quality comes from benchmark scores and cost from OpenRouter pricing.

Failure. This is the highest-confidence refutation. The ratio is dimensionally invalid: quality ϕ_4 is a dimensionless benchmark score (0–1), while cost is market-driven ($/token). Their ratio has no stable physical interpretation. Worse, task-length sensitivity causes $>100\times$ cost prediction errors. A legal document analysis (50K tokens) on GPT-4 costs >$50, while a tweet classification (100 tokens) costs <$0.01. Any fixed-length assumption fails catastrophically. Hidden infrastructure costs (70B models require 8-GPU servers) add further multiplicative errors not captured by per-token pricing.

6.2.4 ϕ_{16}: Instruction Adherence (Refuted, 92%)

Design. Average model M's scores across IFEval (verifiable constraints), MT-Bench (open-ended quality), and AlpacaEval (pairwise preference) to produce a single instruction-following score.

Failure. Instruction adherence is not a single dimension. IFEval measures whether specific verifiable constraints are satisfied (word count limits, format requirements). MT-Bench and AlpacaEval measure stylistic quality and coherence. These create conflicting optimization signals: a model may excel at verifiable constraints while producing stilted prose, or write beautifully while ignoring explicit format requirements. Averaging across these benchmarks produces a number that predicts neither behavior reliably. Novel instruction types (legal compliance, domain-specific constraints) have no benchmark coverage at all.

6.2.5 ϕ_1 v2: Constraint Filter Redesign (Refuted, 85%)

Design. Replace the original token-counting approach with a compositional parser that checks static capability matrices from model documentation (context window, supported formats, available tools).

Failure. Static capability matrices systematically overstate actual capabilities. Model documentation reports maximum context windows, not effective context windows (where quality degrades). Public documentation reflects marketing incentives, not measured performance [Liu et al., 2024]. Compositional constraints ("output JSON with nested arrays where each element follows schema X") require understanding the model's reasoning capabilities, which documentation does not capture.

6.2.6 ϕ_5 v2: Context Fitness Redesign (Refuted, 92%)

Design. Replace embedding cosine similarity with a topic classifier: classify task T by topic, look up model M's historical success rate on that topic from benchmark data.

Failure. Benchmark scores measure broad domain knowledge, not task-specific fitness. A model scoring well on MMLU-medical does not necessarily handle a specific clinical reasoning task well. Static benchmarks also suffer from data contamination: modern LLMs are trained on benchmark datasets, in-

flating scores beyond true capability [Deng et al., 2023]. Domain classification errors from ϕ_2 propagate catastrophically into ϕ_5, compounding the measurement failure.

6.2.7 ϕ_{13} v2: Reasoning Depth Redesign (Refuted, 85%)

Design. Analyze GSM8K solution step counts per model. Use step count as a proxy for multi-step reasoning capability. Generalize to other task types.

Failure. Surface features (mathematical operator count, question length) correlate poorly with actual reasoning depth. The task "prove $\sum_{k=1}^{n}(2k-1) = n^2$" has few operators but requires inductive reasoning. GSM8K performance does not generalize to non-mathematical reasoning: legal argument construction, scientific hypothesis testing, and strategic planning require qualitatively different reasoning structures. Normalization across GSM8K, MATH, and BBH introduces further artifacts from incompatible evaluation scales.

6.2.8 ϕ_{15} v2: Context Window Utilization Redesign (Refuted, 92%)

Design. Fit degradation curves from RULER benchmark data. For a given model M and context length ℓ_T, interpolate the expected performance from the fitted curve.

Failure. Task-type performance variance dominates context-length effects. RULER shows >30% performance gaps between retrieval and question-answering tasks at identical context lengths. A single degradation curve per model averages over these gaps, destroying the task-conditional signal. Positional sensitivity compounds the problem: the "Lost in the Middle" phenomenon [Liu et al., 2024] shows up to 60% performance drops when critical information sits in the context middle versus the ends. The available data (5–7 context length measurement points per model) is insufficient to fit the resulting non-monotonic, task-dependent, position-sensitive degradation surface.

6.2.9 ϕ_4: Performance Predictor (Inconclusive, 30%)

Design. Use the full normalized benchmark database (2.76M records) to predict model M's performance on task type t_T.

Failure. Verification failed due to technical issues (JSON parse errors in the automated verification pipeline). However, the same structural problems affecting all other measurement designs apply: benchmark-to-task domain transfer breakdown, metric normalization artifacts, and data contamination. This phi function has the richest data backing (15,176 models) yet could not be verified, suggesting the problem is fundamental rather than data-limited.

6.3 Root Cause Analysis

The 9 failures share five structural root causes that are unlikely to be resolved by collecting more data or improving normalization algorithms. These failures instantiate a well-known pattern: when a measure becomes a target, it ceases to be a good measure [Goodhart, 1975, Campbell, 1979]. Manheim and Garrabrant [2019] taxonomize four variants of this phenomenon (regressional, extremal, causal, adversarial), all of which appear in our measurement attempts.

1. Metric Scale Incomparability. Public benchmarks use fundamentally different measurement scales: accuracy percentages, Elo ratings, pass@k rates, and pairwise win rates. No statistically sound conversion exists between these scales. Min-max normalization creates comparable numbers but destroys the semantic content that would make them useful for routing.

2. Domain Transfer Breakdown. Performance on curated benchmark tasks does not predict performance on real user tasks. Benchmarks are designed for model comparison, not task-specific prediction. The evaluation conditions (fixed prompts, controlled difficulty, clean data) differ systematically from deployment conditions (varied prompts, uncalibrated difficulty, noisy context).

3. Dimensional Collapse. Complex, multi-dimensional phenomena (reasoning depth, instruction adherence, cost efficiency) cannot be reduced to single scalar scores without losing the task-conditional structure that routing requires. A model's reasoning capability is not a number; it is a function of task type, problem structure, and prompting strategy.

4. Benchmark Contamination. Modern LLMs are trained on datasets that include benchmark test sets, inflating scores beyond true capability [Deng et al., 2023]. This contamination is not uniformly distributed: some models are more contaminated than others, some benchmarks more leaked than others. The resulting scores carry systematic bias that normalization cannot remove.

5. Sparse Measurement vs. Complex Phenomena. The available benchmark data provides sparse coverage (5–7 measurement points for context degradation, single accuracy numbers for domain performance) of phenomena that require dense, multi-dimensional characterization. Fitting complex surfaces to sparse data produces unreliable interpolation at exactly the operating points routing decisions depend on.

6.4 The Gap is the Finding

The measurement gap is a central empirical finding, but it requires careful scoping. The gap applies specifically to *hand-designed, interpretable* phi functions that require explicit metric normalization, domain classification, and semantic mapping.

- **Theory**: 14 of 14 mathematical results verified at 85–95% confidence. The S(M,T) equation converges, has bounded regret, and its parameters are identifiable.
- **Hand-designed measurement**: 0 of 14 measurement designs verified. Every attempt to compute interpretable phi functions from public benchmark data failed due to structural (not data-quantity) limitations.
- **Learned measurement**: The BilinearPhiEngine (Section 8) achieves 83.63% routing accuracy on RouterBench and AUC 0.8006 on RouteLLM by learning phi functions end-to-end from the *same* benchmark data. The gap is not in the data. It is in the requirement that phi functions be interpretable.

This distinction reframes the finding. The binding constraint on LLM routing quality is not the routing algorithm, and it is not the data. It is the demand for interpretable, semantically labeled compatibility functions. When that demand is relaxed, public benchmark data is sufficient. When it is maintained, the five root causes above make static measurement intractable.

6.5 Implications for the Field

The measurement gap, and its learned resolution, have four consequences for LLM routing research:

Public benchmarks are insufficient for interpretable routing. The 7 datasets we ingested represent the most comprehensive public collection available for routing research. If 2.76M records across 15,526

models cannot ground even one hand-designed phi function, the problem is structural for interpretable approaches. Routing-specific benchmarks, ones that measure task-conditional, model-specific performance under deployment conditions, would help.

Learned routing functions bypass the gap. Section 8 demonstrates that bilinear forms trained on pairwise routing data achieve competitive accuracy without requiring explicit metric normalization or domain classification. The normalization and domain transfer problems disappear because the model learns whatever internal representation best predicts routing outcomes. This suggests learned approaches may be more practical than hand-designed ones for production routing.

Online learning is necessary, not optional. Since static benchmark data cannot reliably initialize interpretable phi functions, the router must learn from its own routing decisions. The Robbins-Monro framework (Section 4) with entropy regularization provides the theoretical foundation for this learning. The seed weights serve as an informed prior, not as ground truth.

Transparency about measurement quality matters. Routing papers that report benchmark-grounded scores without documenting the normalization decisions, domain transfer assumptions, and contamination risks are overstating their confidence. The confidence column in Table 1 (Section 3), distinguishing data-grounded from prior-based phi functions, is not a weakness of our framework but a design feature.

7 Empirical Grounding

We describe the data infrastructure supporting the S(M,T) framework: the ingestion pipeline, normalized database, and seed weight computation. While Section 6 establishes that public benchmarks cannot reliably ground individual phi functions, the aggregate data still informs the seed weight vector $\mathbf{w}_0$ and provides the initial endpoint rankings used during the cold-start period.

7.1 Data Ingestion Pipeline

The pipeline processes 7 public datasets through four stages:

1. **Download.** Fetch raw data from Hugging Face Hub, GitHub, and public APIs. Datasets range from 966 records (AlpacaEval) to 1.9M records (Open LLM Leaderboard).
2. **Normalize.** Parse heterogeneous formats (CSV, JSON, Parquet) into a unified schema: `(model_id, benchmark, metric, score, timestamp)`. Apply min-max normalization within each benchmark to produce $[0, 1]$-bounded scores.
3. **Map to Phi.** Route normalized records to the phi functions they inform, following a rule table that maps (dataset, metric) pairs to phi function IDs. For example, MMLU accuracy records map to ϕ_4 (performance predictor) and ϕ_2 (domain classifier), while pairwise comparison data maps to ϕ_8 (cascade position).
4. **Seed Weights.** Compute the initial weight vector $\mathbf{w}_0$ using Equation 2, combining literature priors with data quality multipliers.

The pipeline is idempotent: re-running it on the same raw data produces identical outputs. Total processing time is under 5 minutes on a single machine.

7.2 Normalized Database Statistics

The normalized SQLite database contains 2,764,077 records across 15,526 unique model identifiers. Table 5 shows the data coverage per phi function.

Table 5. Data coverage per phi function category.

Category	Count	Records	Models	$\sum w_i$
Data-grounded	8	2,764,077	15,526	0.847
Computed	3	5,600	350	0.095
Prior-based	3	0	0	0.024
Runtime/Derived	2	0	0	0.024
Total	16	2,764,077	15,526	1.000

The seed weight allocation reflects data quality: data-grounded phi functions receive 84.7% of total weight, while prior-based and runtime functions share the remaining 15.3%. This is a deliberate design choice, not a bug. The weight learner (Section 4) will adjust these weights as online feedback accumulates, potentially upweighting prior-based functions if they prove informative in practice.

7.3 Seed Weight Computation

The seed weights derive from two independent signals (Equation 2):

Literature Priors. We surveyed three routing papers (FrugalGPT [Chen et al., 2023], RouteLLM [Ong et al., 2024], and RouterBench [Hu et al., 2024]) and assigned importance scores on a 0.5–5.0 scale. Performance prediction (ϕ_4, score 5.0) and historical success (ϕ_6, score 4.0) consistently rank highest across all three papers. Cost efficiency (ϕ_{14}, score 2.5) and instruction adherence (ϕ_{16}, score 2.0) receive moderate ratings.

Data Quality Multipliers. Each phi function receives a quality multiplier based on its data coverage:

- High ($q = 1.0$): >1,000 records and >50 models
- Medium ($q = 0.5$): Moderate coverage
- Low ($q = 0.25$): Limited coverage
- None ($q = 0.1$): No direct data, prior-based only

The product $p_i \cdot q_i$, after simplex normalization, produces the seed weights reported in Table 1. The top-4 weights ($\phi_4 = 0.176$, $\phi_6 = 0.141$, $\phi_8 = 0.124$, $\phi_3 = \phi_{13} = 0.106$) together account for 65% of total weight.

7.4 Router Implementation

The v0.1 router implements the full S(M,T) pipeline across 13 heterogeneous endpoints:

- 7 LLM endpoints (Claude Opus 4, Claude Sonnet 4, GPT-4o, DeepSeek R1, DeepSeek V3, Qwen3 Coder, Gemini 2.5 Pro)
- 3 Agent endpoints (auto-search, mathematical verification, first-principles thinking)
- 1 Script endpoint (session capture)
- 2 MCP tool endpoints (Notion, Google Workspace)

This heterogeneous endpoint set distinguishes our implementation from prior routing work, which focuses exclusively on LLM-to-LLM routing. The gate layer handles type-specific constraints (agents have no context window limit, scripts have fixed capabilities, MCP tools have specific supported operations), while the phi functions provide unified scoring across all endpoint types.

7.5 Smoke Test Results

We verify correctness through four routing scenarios (Table 6).

Table 6. Smoke test results demonstrating correct routing behavior.

Query	Selected	Verification
"Write a Python sort function"	DeepSeek V3	Cheapest coding-capable model
"Solve $x^2 + 3x - 4 = 0$" (thinking required)	DeepSeek R1	8/13 endpoints filtered for lacking thinking support
"Explain quantum entanglement"	Claude Sonnet 4	Top general-knowledge model
List endpoints	13 registered	7 LLM + 3 agent + 1 script + 2 MCP

These tests verify gate filtering (thinking requirement eliminates 8 endpoints), phi function evaluation (coding task favors code-optimized models), and cost sensitivity (DeepSeek V3 selected over more expensive alternatives for a straightforward coding task).

8 Learning Phi Functions: The BilinearPhiEngine

Section 6 established that hand-designed phi functions cannot be grounded in public benchmark data due to structural problems: metric incomparability, domain transfer breakdown, and dimensional collapse. This section presents an alternative: learning the phi functions end-to-end from pairwise routing data, bypassing the measurement gap entirely.

8.1 Motivation

The measurement gap arises because hand-designed phi functions impose semantic structure ("domain classifier," "reasoning depth") that requires mapping heterogeneous benchmark metrics into a common space. This mapping is where all 9 designs failed. But routing does not require interpretable phi functions. It requires phi functions that *predict routing outcomes*. If we replace the 16 semantic phis with 16 learned bilinear functions trained directly on routing-relevant data, the normalization and domain transfer problems disappear: the model learns whatever internal representation best predicts scores, without requiring us to specify what each dimension means.

Learned representations for routing are not new. RouteLLM [Ong et al., 2024] uses matrix factorization over preference data, an approach with well-established foundations in collaborative filtering [Koren et al., 2009]. RouterDC [Chen et al., 2024] trains dual contrastive embeddings. EmbedLLM [Zhuang et al., 2024] learns compact model representations for performance prediction. GraphRouter [Feng et al., 2024] constructs heterogeneous task-model graphs. Our contribution here is not the bilinear form itself, but its role as a drop-in replacement for the hand-designed phi functions within the S(M,T) architecture: preserving the framework's multiplicative gating and convergence properties while bypassing the measurement gap.

8.2 Architecture

The BilinearPhiEngine instantiates the weighted compatibility term of S(M,T) as a set of learned bilinear forms:

$$\hat{\phi}_i(M,T) = \sigma\left(\mathbf{e}_T^\top \mathbf{W}_i \, \mathbf{e}_M + b_i\right), \quad i = 1,\ldots,K \tag{10}$$

where $\mathbf{e}_T \in \mathbb{R}^{768}$ is a frozen prompt embedding from a pre-trained sentence encoder (all-mpnet-base-v2), $\mathbf{e}_M \in \mathbb{R}^{256}$ is a learned model embedding, $\mathbf{W}_i \in \mathbb{R}^{768\times256}$ is a learned bilinear interaction matrix, $b_i \in \mathbb{R}$ is a bias term, and σ is the sigmoid function. The overall score combines these via softmax-weighted sum:

$$\hat{S}(M,T) = \sum_{i=1}^{K} \alpha_i \cdot \hat{\phi}_i(M,T), \quad \alpha_i = \frac{e^{a_i}}{\sum_j e^{a_j}} \tag{11}$$

where $\mathbf{a} \in \mathbb{R}^K$ are learned logits producing weights on the probability simplex. With $K = 16$ bilinear heads, the architecture has approximately 3.15 million trainable parameters:

- 16 bilinear matrices $\mathbf{W}_i$: $16 \times 768 \times 256 = 3{,}145{,}728$ parameters
- 16 biases b_i: 16 parameters
- 48 model embeddings $\mathbf{e}_M$: $48 \times 256 = 12{,}288$ parameters
- 16 head weight logits a_i: 16 parameters

The prompt encoder is frozen during training. Only model embeddings, bilinear matrices, head weights, and biases are learned. This separation is deliberate: the prompt encoder provides a fixed, high-quality text representation; the bilinear matrices learn how different models interact with different prompt features.

Diversity Regularization. To prevent the 16 heads from collapsing to identical functions, we add a cosine-similarity penalty between head weight vectors:

$$\mathscr{L}_{\text{div}} = \frac{1}{K(K-1)} \sum_{i\neq j} \left|\cos(\text{vec}(\mathbf{W}_i), \text{vec}(\mathbf{W}_j))\right| \tag{12}$$

where $\text{vec}(\mathbf{W}_i)$ flattens the bilinear matrix into a vector. The total loss is $\mathscr{L} = \text{MSE}(\hat{S}, S^*) + \lambda_{\text{div}} \cdot \mathscr{L}_{\text{div}}$ with $\lambda_{\text{div}} = 0.01$.

8.3 Training Data

We unify five public routing datasets into a single training corpus of 740,803 (prompt, model, score) triples:

A canonical model registry maps 180 model name aliases (e.g., “gpt-4-1106-preview,” “gpt4-turbo,” “openai/gpt-4-1106”) to 48 canonical model identifiers, each receiving a learned 256-dimensional embedding. Scores are normalized to $[0,1]$ per source. Data is split 80/10/10 into train/validation/test sets with a fixed random seed.

Table 7. Training data sources with canonical model resolution.

Source	Samples	Models	Score Type
RouterBench	401,467	11	Accuracy (0/1)
RouteLLM	218,202	2	Win rate (0/1)
Arena 55K	111,712	45	Elo-derived (continuous)
MT-Bench	6,710	5	Rating (1–10, normalized)
RewardBench	2,712	31	Accuracy (0/1)
Total	**740,803**	**48**	

Prompt Encoding. The 162,422 unique prompts are pre-encoded using the raw transformer model (AutoTokenizer + AutoModel from all-mpnet-base-v2) with mean pooling and L2 normalization, producing 768-dimensional embeddings. This one-time encoding step takes approximately 45 minutes on Apple Silicon (MPS); the resulting cache enables the training loop to run at full GPU speed without I/O bottleneck.

8.4 Training Procedure

Training uses AdamW with differential learning rates: 10^{-4} for bilinear matrices $\mathbf{W}_i$, 10^{-3} for model embeddings $\mathbf{e}_M$ and head weights $\mathbf{a}$. Batch size is 256. Early stopping with patience 10 monitors validation MSE loss.

The full model (all 5 sources) trains for 31 epochs before early stopping, reaching best validation loss of 0.1217. Training takes approximately 2 minutes per epoch on Apple M-series hardware (MPS backend), for a total wall time under 70 minutes.

8.5 Benchmark Evaluation

We evaluate the trained BilinearPhiEngine on two established routing benchmarks. A third benchmark (RouterEval) proved incompatible because it evaluates routing among 1,000+ open-source fine-tunes from the Open LLM Leaderboard, a model pool completely disjoint from our 48 API models.

8.5.1 RouterBench

RouterBench [Hu et al., 2024] provides 73,008 test instances across 11 model pairs, split into 0-shot and 5-shot settings. For each instance, the router must select the model most likely to answer correctly.

Table 8. RouterBench evaluation results.

Method	0-shot Acc.	5-shot Acc.	Overall
Random baseline	55.50	64.34	–
Always-strong	84.35	86.66	85.51
BilinearPhi (ours)	**80.49**	**86.76**	**83.63**
Oracle (upper bound)	96.13	96.76	96.45

The model achieves 83.63% overall accuracy. In the 5-shot setting, it slightly exceeds the always-strong baseline (86.76% vs. 86.66%), indicating it learns when cheaper models suffice. The router selects from 11 distinct models, not just the strongest, demonstrating genuine multi-model routing rather than defaulting to a single endpoint.

8.5.2 RouteLLM

RouteLLM [Ong et al., 2024] evaluates binary routing (strong vs. weak model) across MMLU, GSM8K, and MT-Bench subsets. Quality is measured as the average win rate of selected models, with AUC computed across routing thresholds.

Table 9. RouteLLM evaluation results. Quality ratio = quality / always-strong quality.

Threshold	Quality	Quality Ratio
Always weak	0.7187	84.09%
20% strong	0.7609	89.02%
50% strong	0.8065	94.35%
80% strong	0.8399	98.27%
Always strong	0.8547	100.00%
AUC	**0.8006**	
Routing accuracy	94.55%	

At the 50% strong-model threshold (routing half of queries to the cheaper model), the system retains 94.35% of always-strong quality. The AUC of 0.8006 indicates effective quality-cost tradeoff across the full threshold range.

8.5.3 Internal Evaluation

On the held-out test split (10% of training data, 74,080 samples), the model achieves MSE 0.1223, MAE 0.2689, Pearson correlation 0.5305, and binary routing accuracy 74.39%.

8.6 Leave-One-Out Generalization Analysis

To test whether the BilinearPhiEngine learns transferable routing features rather than source-specific patterns, we conduct a leave-one-out ablation study. Five models are trained, each excluding one of the five data sources entirely. Each ablated model is then evaluated on its held-out source (zero-shot generalization) and on all five sources (cross-source transfer).

8.6.1 Training Dynamics

Table 10. Leave-one-out ablation: training summary. Correlation is Pearson r on the held-out source.

Excluded	Train N	Epochs	Val Loss	Heldout r	Heldout MSE
RouterBench (401K)	271,470	21†	0.0752	0.6505	0.0790
RouteLLM (218K)	418,081	17†	0.1623	0.3192	0.1742
Arena 55K (112K)	503,273	30	0.1131	0.4596	0.1794
MT-Bench (7K)	587,275	30	0.1211	0.4690	0.1705
RewardBench (3K)	590,473	30	0.1205	0.3849	0.1539
Full model	592,641	31†	0.1217	–	0.1223

† Early stopped (patience 8).

Two patterns stand out. First, removing RouterBench (54% of data) causes early stopping at epoch 21, while removing smaller sources allows training to run all 30 epochs. This confirms RouterBench's role as the diversity backbone of the dataset. Second, removing MT-Bench (0.9% of data) or RewardBench (0.4%) produces validation loss nearly identical to the full model (0.1211 and 0.1205 vs. 0.1217), indicating these sources add negligible marginal information.

8.6.2 Cross-Source Generalization Matrix

Table 11 presents the full 5×5 cross-source correlation matrix. Each row is a model trained *without* that source; each column is the evaluation target. Diagonal entries (bold) represent zero-shot generalization to an unseen source.

Table 11. Cross-source Pearson correlation matrix. Rows: excluded source during training. Columns: evaluation source. Diagonal (bold) = zero-shot generalization.

Trained w/o	RBench	RLLM	Arena	MTB	RwdB
RouterBench	**0.651**	0.073	0.197	0.174	0.169
RouteLLM	0.425	**0.319**	0.457	0.468	0.385
Arena 55K	0.583	0.321	**0.460**	0.470	0.386
MT-Bench	0.682	0.321	0.459	**0.469**	0.385
RewardBench	0.679	0.320	0.459	0.468	**0.385**

Finding 1: RouterBench is the generalization anchor. When RouterBench is included in training (rows 2–5), the RouterBench column shows correlation 0.43–0.68. When RouterBench is excluded (row 1), cross-source generalization collapses: correlation on all non-RouterBench sources falls to 0.07–0.20. RouterBench's 11-model, diverse-prompt format provides the signal diversity that enables generalization.

Finding 2: RouteLLM is the hardest transfer target. The RouteLLM column is capped at $r \approx 0.32$ regardless of which other source is removed. RouteLLM's binary 2-model setup (GPT-4 vs. Mixtral on academic benchmarks) represents a qualitatively different routing problem that other benchmark formats cannot fully predict.

Finding 3: Small sources are redundant. Rows 4–5 (excluding MT-Bench or RewardBench) produce nearly identical cross-source correlations to each other and to the full model. Training on just RouterBench + RouteLLM + Arena 55K (731K samples) achieves the same generalization quality as the full 741K dataset.

Finding 4: Unseen sources are predictable. The diagonal entries (zero-shot generalization) range from 0.319 to 0.651, confirming that the bilinear architecture learns transferable routing features. RouterBench is the most predictable from other sources ($r = 0.651$), while RouteLLM is the least ($r = 0.319$).

8.7 Connection to the Measurement Gap

The BilinearPhiEngine achieves what the hand-designed phi functions could not: reliable routing scores grounded in public benchmark data. The resolution is architectural. Hand-designed phis require explicit metric normalization, domain classification, and semantic mapping, each of which introduces fragile assumptions (Section 6). The bilinear architecture sidesteps all three by learning the mapping end-to-end. The prompt encoder provides a fixed text representation; the bilinear matrices learn whatever model-prompt interaction structure best predicts routing outcomes.

The tradeoff is interpretability. The 16 learned bilinear heads have no semantic labels. We cannot say "head 3 captures reasoning depth" or "head 7 measures instruction adherence." The hand-designed phi functions in Table 1 remain the interpretable interface for the production router (Section 7), initialized with seed weights and refined through online learning. The bilinear engine provides the empirical validation that the weighted-phi architecture can achieve competitive routing accuracy when the learning problem is well-posed.

8.8 Scaling Ablation: BilinearPhiEngine V3

The BilinearPhiEngine V1 (Section 8) was trained on a single data corpus with a compact architecture (3.15M parameters, $K = 16$ heads, $D_{\text{model}} = 256$). A natural question is whether scaling the architecture with more data, more models, and additional inductive biases improves routing quality. We designed BilinearPhiEngine V3 to test this hypothesis. The answer is no: V3 underperforms V1 on every benchmark.

8.8.1 V3 Architecture and Training

V3 scales three dimensions of V1 simultaneously:

1. **Parameter expansion.** $K = 24$ bilinear heads (vs. 16), $D_{\text{model}} = 768$ (vs. 256), producing bilinear matrices $\mathbf{W}_i \in \mathbb{R}^{768\times768}$. Total parameters: 14.6M (4.63× V1).
2. **Data expansion.** Training on a V2 database (4.0 GB, 3.86M rows from 10 additional preference datasets including UltraFeedback, Nectar, WildBench, HelpSteer2, and HH-RLHF). After filtering unrecognized model IDs (53 of 101 models), 1.98M usable rows with 356,358 unique prompts remain (2.2× the unique prompts in V1).
3. **Architectural additions.** Three new components added via progressive training stages: task-type conditioning embeddings (Stage 3), multi-head cross-attention between prompt and model representations (Stage 4), and metadata feature integration including source and score normalization information (Stage 5).

Training uses a 5-stage progressive schedule on an A100 80GB GPU: (1) frozen prompt encoder with bilinear matrices only, (2) full end-to-end fine-tuning including unfreezing the sentence encoder, (3) task conditioning, (4) cross-attention, (5) metadata features. Each stage initializes from the previous stage's best checkpoint.

8.8.2 Results

We evaluate the two most informative checkpoints: Stage 2 (the first fully trained model, before architectural additions) and Stage 5 (the final model with all additions).

Table 12. V1 vs. V3 on external routing benchmarks. V1 outperforms all V3 variants. Δ is percentage points relative to V1.

	RouterBench			RouteLLM		
Model	**0-shot**	**5-shot**	**Overall**	**AUC**	**Q@50%**	**Rt. Acc.**
V1 (3.15M)	**80.49**	**86.76**	**83.63**	**0.8006**	**0.9435**	**94.55**
V3-S2 (14.6M)	79.18	86.70	82.94	0.7972	0.8020	93.10
V3-S5 (14.6M)	77.44	85.51	81.47	0.7973	0.8022	92.79
Δ (best V3)	−1.31	−0.06	−0.69	−0.0033	−0.1413	−1.45
Δ (worst V3)	−3.05	−1.25	−2.16	−0.0034	−0.1415	−1.76

External Benchmarks. On RouterBench, V3 Stage 2 loses 0.69 percentage points overall, while Stage 5 loses 2.16 points. On RouteLLM, both V3 stages lose approximately 0.003 AUC. The quality-at-50%-strong metric shows a larger gap: V3 retains only 80.2% of always-strong quality at 50% cost (vs. V1's 94.4%), suggesting degraded confidence calibration.

Task-Stratified Analysis. V3 is competitive on math tasks (94.4%), which have unambiguous correctness labels. Performance degrades most on tasks requiring subjective quality judgments: writing

Table 13. RouterBench accuracy by task type. V3-S2 is the best V3 variant. Bold indicates the best result per task.

Task Type	V3-S2 (%)	V3-S5 (%)	*N*
Math	94.39	**94.37**	15,547
Reasoning	81.12	79.03	44,922
Safety	79.79	79.04	1,064
Code	77.80	**78.00**	500
Factual QA	77.95	76.65	6,616
Instr. following	75.55	73.36	687
Conversation	72.88	70.77	520
Writing	68.05	67.67	3,152

(68%), conversation (71–73%), and instruction following (74–76%). This pattern is consistent with the V2 training data containing noisier labels from preference-based datasets.

Internal Evaluation. On the held-out V2 test split (198,304 samples), V3 shows near-zero or negative correlation with ground truth:

Table 14. Internal evaluation on V2 test data. V1 baseline computed on V1 test split.

Model	Pearson r	MSE	MAE	Binary Acc. (%)
V1	0.5305	0.1223	0.2689	74.39
V3-S2	-0.0483	0.1365	0.3188	57.99
V3-S5	-0.0335	0.1357	0.3162	58.09

The internal correlation collapse ($r = 0.53 \rightarrow r \approx -0.04$) is more dramatic than the external benchmark regressions. This divergence has a structural explanation: the V2 database labels incorporate preference-based scores from datasets with different quality semantics than V1's benchmark-accuracy scores. V3 learned the V2 label distribution, but that distribution is not well-aligned with the actual model quality ranking.

8.8.3 Why Scaling Failed: Analysis

Four factors explain V3's underperformance:

1. Label distribution shift. The V2 database merged 10 new preference datasets with the original 5 benchmark sources. Of 101 model IDs in V2, 53 were unrecognized by the canonical registry, causing 1.87M rows (49% of total) to be skipped. The surviving data has different score semantics: preference-based "win rate" labels from UltraFeedback and Nectar encode relative quality between model pairs, not absolute task accuracy. Training on these labels teaches V3 to predict pairwise preferences rather than absolute routing scores.

2. Sparse model coverage. V3's embedding table covers 93 models (vs. V1's 48), but the new models have far fewer training examples. The V2 database has 356K unique prompts spread across 93 models, versus V1's 162K prompts across 48 models. Per-model embedding quality degrades with sparse coverage, and the embedding space becomes undertrained for tail models.

3. Progressive training instability. Each stage inherits and potentially amplifies errors from its predecessor. Stage 5 (the final model) performs worse than Stage 2 (the first fully trained model) on

RouterBench (81.47% vs. 82.94%), confirming that the architectural additions (task conditioning, cross-attention, metadata) degraded rather than improved routing quality. This is characteristic of overfitting to training artifacts rather than learning generalizable routing features.

4. Architectural overparameterization. With 14.6M parameters trained on 1.98M usable samples, V3 has a parameter-to-sample ratio of 7.4:1. V1's ratio is 4.3:1 with cleaner labels. The bilinear matrices $\mathbf{W}_i \in \mathbb{R}^{768\times768}$ have 590K parameters each (vs. V1's 197K), providing excess capacity that absorbed noise in the V2 labels rather than learning better routing representations.

8.8.4 Implications

The V3 ablation establishes that the original V1 architecture is not capacity-limited. The bilinear routing problem, at least at the current scale of 48–93 models, does not benefit from larger embedding dimensions, more bilinear heads, or additional architectural components. V1's 3.15M parameters with clean labels outperform V3's 14.6M parameters with noisy labels on every metric. This result has two practical implications.

First, **data quality dominates model capacity** for learned routing. The path to better routing accuracy runs through cleaner, more consistent training labels, not larger models. This aligns with the measurement gap finding (Section 6): the bottleneck is grounding quality signals, not the capacity of the model consuming those signals.

Second, **progressive architectural additions provide diminishing returns** when the base training signal is noisy. Task conditioning, cross-attention, and metadata features all assume the training labels contain structured information that can be decomposed along these axes. When labels are inconsistent, these additions fit noise rather than signal.

The V3 experiment thus serves as a controlled counter-test to the V1 results. V1's performance is not an accident of small-scale evaluation. A systematic attempt to improve it through data scaling, parameter scaling, and architectural enrichment produced consistently worse results. The compact bilinear architecture, trained on well-curated data, remains the strongest configuration.

9 Production Validation: Sixty-One Days of Shadow Routing

Section 8 reported 83.63% accuracy on RouterBench using a trained bilinear architecture (Bilinear-PhiEngine). That number is the benchmark headline. This section reports a second result from a different question: when the S(M,T) framework with seed-prior weights and online Robbins-Monro learning is deployed in production over an extended period, what do we observe? We answer with 1,580 routing decisions collected across 61 days (March 20–May 24, 2026) of continuous shadow routing on a working developer's prompt stream.

We report four findings, two of which are bugs discovered during this paper's empirical audit. The bugs are part of the result: they characterize what shipping a learner in production actually looks like.

9.1 Deployment

The shadow router (`scripts/shadow_router.py`, v2.4) is hooked into a Claude Code session via the `UserPromptSubmit` event. Every prompt the user submits is independently routed by the S(M,T) framework before the prompt is executed. The router's decision is logged to a SQLite database; the actual execution proceeds through the unchanged production path (`claude-opus-4` as the always-on baseline). This shadow-mode design lets us collect routing decisions and cost-savings estimates without

exposing the user to any potential quality regression.

The endpoint pool contains 344 LLMs across 56 providers plus 5 local agents (auto-search, thinking, ramanujan, session-capture, index-research). Of the 16 phi functions, the production system uses the fast scoring path described in Section 3: a Boltzmann cost-decay form $e^{-\beta \cdot r}$ with $\beta = 0.10$ multiplied by a capability–complexity product. Weights are initialized from the literature-prior seed (Section 7, ű48) and updated online via Robbins-Monro after each session via the outcome collector (`scripts/outcome_collector.py`

9.2 Finding 1: 94.7% Cost Reduction Against Always-Opus (Honest Accounting)

Table 15 summarizes the cost reduction observed across 1,580 decisions. The baseline is the cost the system would have incurred had every prompt been routed to `claude-opus-4` (current production default). The routed cost is what the router selected, with agent-subprocess costs counted explicitly (Section 9.3).

Table 15. Production cost reduction over 61 days of shadow routing, with agent-subprocess costs included.

Metric	Value (USD)
Baseline (always-Opus)	$99.66
Routed (S(M,T) selection)	$5.30
Total savings	$94.36
Reduction	94.68%
Savings per 1,000 prompts	$59.72

The reduction is not driven by a single optimization. It compounds across three orthogonal mechanisms:

1. **Free-tier routing.** 842 of 1,580 decisions (53.3%) selected the `minimax-m2.5:free` endpoint; an additional 164 (10.4%) went to `stepfun-step-3.5-flash:free`; 79 (5.0%) went to `hermes-3-llama-3.1-405b:free`. Together: 68.7% of all production decisions routed to free OpenRouter tiers.
2. **Local agent routing.** 402 of 1,580 decisions (25.4%) selected a local agent (auto-search, thinking, or ramanujan). These have non-zero subprocess cost ($0.010–$0.030 per invocation, summing to $5.30 across all agent decisions) but remain far cheaper than the always-Opus baseline.
3. **Selective premium use.** Only 56 of 1,580 decisions (3.5%) selected `claude-sonnet-4` or another paid model; the rest were free or near-free.

The 25.4% local-agent fraction is the empirical realization of the heterogeneous routing thesis. A router that scored only LLMs would route those decisions to LLMs, paying full `claude-opus-4` prices for capability already present in the agent registry.

9.3 Agent-Subprocess Cost Correction

An earlier draft of this section reported a 99.99% cost reduction. That figure was the result of a production-code accounting shortcut: `route_prompt_fast` treated every agent dispatch as zero-cost, on the (false) assumption that local agents have no token cost. In reality, four of the five local agents invoke downstream LLMs to fulfill their tasks: `auto-search` uses DeepSeek-V3, `thinking` uses DeepSeek-R1, and `ramanujan` uses DeepSeek-Prover-V2-671B. Their per-invocation costs are non-zero ($0.010–$0.030).

We discovered this during empirical audit for this paper and applied a medium-grade fix: each agent now has a historical-average per-invocation cost (Table 16), used in both the scoring function (`_score_agent`)

and the routed-cost logging path (`route_prompt_fast`). All 402 historical agent decisions were re-costed using these averages.

Table 16. Per-agent average invocation cost, derived from downstream LLM usage.

Agent	Downstream LLM(s)	Avg Cost (USD)
auto-search	DeepSeek-V3 ($0.28/$0.42 per M)	$0.010
ramanujan	DeepSeek-Prover-V2-671B, DeepSeek-R1	$0.015
thinking	DeepSeek-R1 ($0.70/$2.50 per M)	$0.030
session-capture	None (local regex)	$0.000
mcp-* (Notion, Google)	None (API only)	$0.000

This finding is significant beyond the headline number. The deterministic cost factor $e^{-\beta \cdot \text{cost_rank}}$ used in the V1 production code is wrong in principle for agents: their cost is a distribution over possible downstream LLM invocations, not a fixed scalar. The V2 framework refinement (Section 3 ű V2) addresses this by replacing the deterministic cost factor with the CVaR-based stochastic cost term

$$C(M,T) = \frac{1}{1+\beta \cdot \text{CVaR}_{0.99}[C]}.$$

Agents are the canonical case where this matters. A short-term fix (per-agent historical averages) restores honest accounting; the structural fix is the V2 stochastic cost term.

9.4 Finding 2: Coding-Dominant Workload

The task-type distribution (Table 17) is heavily skewed toward coding (61.8%) and knowledge retrieval (19.8%). This is consistent with the prompt stream of a software engineer using Claude Code as a daily driver. We make no claim about generalization to other workloads; the routing distribution we observe is conditional on this workload, and any benchmark that does not match this conditional distribution will produce different routing patterns.

Table 17. Task-type distribution across 1,580 production decisions.

Task type	Decisions	%
coding	976	61.8
knowledge	313	19.8
reasoning	97	6.1
general	72	4.6
long_context	47	3.0
instruction_following	46	2.9
math	29	1.8

This is a real but mild generalization concern. We discuss its implications for paper claims in Section 12.

9.5 Finding 3: Two Production Bugs Discovered During Empirical Audit

In preparing this section, we re-examined the production learner state. The `learned_weights.json` file reported a decision count of 11, despite 1,328 outcome signals having been collected. Investigation surfaced two production bugs that materially affect what the published BilinearPhiEngine results say about learning behavior in deployment. We document both: they are not incidental implementation defects but characterize the gap between batch-trained academic numbers and online-trained production behavior.

Bug 1: Decision-count not persisted across restarts. The `WeightLearner` class loads its weights from disk via `load_learned_weights()` on instantiation but hardcodes `self._decision_count = 0`. The persisted JSON file contained the correct count, but the loader silently dropped it. Consequence: every session restart reset the Robbins-Monro step-size schedule to $\alpha_1 = 1/(1+1) = 0.5$, causing the first update of each session to perturb weights by 50% rather than the cooled $1/(n+1)$ rate. The expected stochastic-approximation convergence (Section 4) requires a non-resetting cooling schedule. Without it, the learner re-cold-starts on every session, repeatedly over-weighting whichever signal arrives first.

Bug 2: Fast-path routing omits the phi breakdown. The production hook (`route_prompt_fast`) returns `phi_breakdown: {}` for every decision in the interest of latency (target < 85 ms). Only the diagnostic full-engine path (`route_prompt_full`) computes the per-phi scores needed by the learner. Of 1,197 outcome signals passing the confidence gate, only 52 (4.3%) joined a routing decision with a populated breakdown. The remaining 1,145 signals contained quality information but could not produce a gradient.

Both bugs reflect a tension that is general to deploying learned routers: the latency budget that justifies a fast scoring path conflicts with the diagnostic logging required to support online learning. The medium fix we apply (Appendix **??**) reconstructs an approximate phi breakdown from data already computed in the fast path. Warm-cache latency remains $\sim$0.6 ms; gradient signal is restored on every decision.

9.6 Finding 4: Replay Confirms the Measurement Gap

To recover what the learner *would* have produced if the bugs had not been present, we replay all 1,197 confidence-gate-passing outcome signals through the fixed Robbins-Monro learner offline. Of these, 52 join to non-empty phi breakdowns (the diagnostic full-engine path was used for those). We feed only the 52 signals with usable gradients. Initial weights are the literature-prior seed.

Table 18 shows the result. Twelve of sixteen phi functions converged to zero. Four phis concentrated. The mass moved away from the literature priors and toward four specific phis: cross-node capability, complexity estimation, cost efficiency, and uncertainty.

Table 18. Seed vs. replay-learned weights after 52 updates on production outcome signals.

Phi	Name	Seed	Replay	Δ
ϕ_{10}	cross_node_capability	0.008	**0.410**	+0.402
ϕ_3	complexity_estimator	0.106	**0.366**	+0.260
ϕ_{14}	cost_efficiency_ratio	0.088	**0.169**	+0.081
ϕ_7	uncertainty_score	0.016	**0.055**	+0.039
ϕ_4	performance_predictor	0.176	0.000	–0.176
ϕ_6	historical_success	0.141	0.000	–0.141
ϕ_8	cascade_position	0.123	0.000	–0.123
ϕ_{13}	reasoning_depth	0.106	0.000	–0.106
ϕ_1	constraint_filter	0.071	0.000	–0.071
ϕ_{16}	instruction_adherence	0.071	0.000	–0.071
ϕ_2	domain_classifier	0.035	0.000	–0.035
(others)	—	various	0.000	—

The literature priors emphasized predicted accuracy (ϕ_4, prior 5.0), historical success (ϕ_6, prior 4.0), and cascade position (ϕ_8, prior 3.5) as the three most important phi functions for routing. The production data rejects all three. The phis that concentrated were either deprioritized in the literature (cross-node

capability, prior 0.5) or moderately prioritized (complexity estimator, prior 3.0; cost efficiency, prior 2.5).

This is the measurement gap (Section 6) made concrete from production data. Benchmark-derived priors over-weight phi functions whose signal degrades under domain transfer. The phi functions that survive 52 updates of real outcome feedback are those measurable directly from the routing context itself: *is this endpoint a local agent or an LLM* (cross-node), *how complex is the task* (complexity estimator), *how much will this cost* (cost efficiency), *how confident is the classifier* (uncertainty).

We do not claim 52 updates is sufficient for convergence. The Robbins-Monro analysis (Section 4) requires substantially more samples for any tight convergence claim, and even after the bug fix, the production stream will need additional weeks to produce a defensible learning curve. What we do claim: a single-phase, batch-bilinear architecture trained on RouterBench (Section 8) and an online-scalar learner deployed on production outcomes both point at the measurement gap as the practically dominant problem in real-world routing. The benchmark and the production data agree.

9.7 What Production Validates and What It Does Not

The production deployment validates three claims of this paper:

- **Heterogeneous routing is operational.** 25.6% of production decisions selected local agents, not LLMs. The framework treats both within a unified scoring algebra.
- **Multiplicative gating is durable.** Over 61 days, no decision violated boundedness or hit a divide-by-zero path. The framework's $[0, 1]$ score invariants held.
- **The measurement gap is empirical, not just adversarial.** The phi functions that the literature ranks highest collapse fastest under real outcome feedback.

The deployment does not validate:

- **Convergence rate.** The Robbins-Monro convergence proof requires more decisions than we have to make a tight claim. We make none.
- **Workload generalization.** The 61.6% coding skew limits any claim about routing distributions outside software-engineering workloads.
- **Quality preservation.** The outcome signals score 89.5% in the *good* bucket and 6.4% in the *high* bucket. This is consistent with the routed endpoints being adequate, but the signals are proxies (acceptance, continuation, error-flow), not ground truth.

These limitations are honest. The cost-reduction headline is supported. The convergence headline is not yet supported. Section 12 discusses what production-scale validation would look like next.

9.8 Three Views of the Same 1,593 Prompts

A natural question, given that Section 8 reports a benchmark accuracy of 83.63% for the trained bilinear architecture (BilinearPhiEngine) and this section reports a 94.7% production cost reduction for the scalar router with online learning: which router would have been better in production? We answer by feeding all 1,593 historical production prompts through the trained BilinearPhi model offline and comparing its choices against the V1 scalar router's actual decisions.

Methodology. For each production prompt, we encode it with the same frozen sentence-transformer (`all-mpnet-base-v2`) BilinearPhi was trained on, then score all 48 canonical models BilinearPhi knows. We take its top-1 choice and map it to the nearest production endpoint via a family-substring heuristic (`gpt-4` family → `openai/gpt-4`, etc.). We then compute the counterfactual cost under that choice using mid-tier paid pricing as a conservative estimate. We record the agreement rate with V1 and the cost differential.

Three limitations we accept as findings.

1. **Vocabulary mismatch.** BilinearPhi was trained on 48 canonical models, most of which are first- and second-generation (`gpt-4`, `claude-2`, `mistral-medium`, `wizardlm-13b`). The production pool of 344 LLMs is dominated by newer free OpenRouter tiers (`minimax-m2.5:free`, `stepfun-step-3.5-flash:free`) that BilinearPhi has no embeddings for.
2. **Agent blindness.** BilinearPhi has zero embeddings for local agents (auto-search, thinking, ramanujan). On the 402 production prompts that V1 routed to an agent (25.2% of all decisions), BilinearPhi structurally cannot make the same choice.
3. **No ground truth.** We measure agreement with V1 and counterfactual cost, not "accuracy." Production prompts have no ground-truth labels.

Results. Table 19 summarizes the comparison.

Table 19. V1 scalar (executed) vs. BilinearPhi (offline replay) on the same 1,593 production prompts.

Metric	V1 scalar	BilinearPhi (offline)
Decisions evaluated	1,593	1,593
Baseline cost (always-Opus, USD)	$100.17	$100.17
Routed cost (USD)	$5.30	$92.22
Cost reduction vs. Opus	**94.71%**	**7.94%**
Agent picks	402 (25.2%)	0 (structural)
Median decision latency	0.55 ms	∼30–50 ms
Agreement rate with V1 actual	—	0.0%

Reading the result. BilinearPhi—the model that achieves 83.63% accuracy on RouterBench—would have spent $92.22 routing the same 1,593 production prompts that V1 scalar+RM routed for $5.30. Cost reduction collapses from 94.7% to 7.9%. The agreement rate is 0% because the two routers operate over essentially disjoint endpoint vocabularies: BilinearPhi knows 48 benchmark-trained canonical models; the production pool is dominated by 296 newer free-tier models and 5 local agents that BilinearPhi was never trained on. Section 10 addresses this gap via an Endpoint Profiler agent that bootstraps bilinear embeddings for previously-unseen endpoints, raising bilinear's cost reduction to 51.63%.

This is not a bilinear-vs-scalar comparison in any clean sense. It is a *benchmark-trained vs. production-deployed* comparison. The bilinear model is the more accurate router on the corpus it was trained on. The scalar+RM router is the more cost-effective router on the workload it is actually serving. The two answer different questions and we report both.

The frame this clarifies. A reviewer or practitioner asking "which router should I use?" is asking the wrong question. The correct framing is two-axis:

- **If you have ground-truth labels for your prompt distribution and a fixed model pool that**

overlaps with public benchmarks: train the bilinear. You will extract 83.63% accuracy and a defensible Pareto frontier on cost-quality.

- **If you have a heterogeneous endpoint pool that includes agents, scripts, or frontier-fresh free models the benchmarks do not cover**: deploy the scalar with online learning. You will route 25%+ of traffic to non-LLM endpoints and capture a 90%+ cost reduction that the bilinear architecturally cannot replicate.

The S(M,T) framework supports both. The V2 refinements (Section 3.7) bring the production scalar's accounting up to the standard the bilinear naturally satisfies on its narrower corpus. Future work—particularly the V2 stochastic cost term applied to bilinear embeddings of agent endpoints, and the V2 type dispatch giving agents first-class phi vocabulary—would let a single architecture inherit the strengths of both. This is V3 territory.

10 Endpoint Profiler: Bootstrapping Bilinear Embeddings via Feature Synthesis

Section 9 (Three Views, Table 19) reported a 0% agreement rate between V1 scalar routing and the offline-replayed BilinearPhi on the same 1,593 production prompts. The cause was vocabulary mismatch: BilinearPhi knew 48 canonical models from its RouterBench training corpus, while production routed across a pool of 344 LLMs and 5 local agents, with zero overlap. This section presents the *Endpoint Profiler*: an agent that bootstraps 256-dimensional bilinear embeddings for previously-unseen endpoints via feature synthesis, closing the vocabulary gap without retraining the bilinear architecture.

10.1 The Bootstrapping Problem

The trained BilinearPhi engine computes phi scores as

$$\hat{\phi}_i(M,T) = \sigma\left(\mathbf{e}_T^\top \mathbf{W}_i \mathbf{e}_M + b_i\right)$$

where $\mathbf{e}_M \in \mathbb{R}^{256}$ is a learned model embedding. For canonical models in the training corpus, this embedding was learned end-to-end. For any new endpoint (a recently released free-tier model on OpenRouter; a local agent; a fine-tuned variant), there is no $\mathbf{e}_M$. The bilinear architecture cannot score the endpoint at all.

A naive solution is to retrain the bilinear engine on a corpus that includes the new endpoints. This requires labeled data, retraining time, and is impossible for the heterogeneous endpoints (local agents, scripts) where no benchmark corpus exists. We instead propose to *synthesize* the missing $\mathbf{e}_M$ from observable features.

10.2 Method: Three-Stage Synthesis

The Endpoint Profiler operates in three stages.

Stage 1: Feature Extraction. For each endpoint, the Profiler extracts an 80-dimensional feature vector $\mathbf{f}_M$ combining structural and behavioral information:

- **Structural features (40 dims)**: 7-dim capability profile, parameter count (log), context length (log), tool/vision support, family one-hot (10), provider one-hot (10), and 7 benchmark scores (MMLU, HumanEval, GSM8K, etc.) when available.
- **Judge features (40 dims)**: estimated by an LLM-as-judge (DeepSeek-R1) reading the model card,

name, and metadata, then rating the model across 40 capability and behavioral dimensions on a 0–1 scale.

The LLM-as-judge stage was executed on all 398 endpoints (344 production LLMs + 5 agents + 48 canonical references for validation). Total spend was $2.31. The judge produces consistent estimates across known reference models (GPT-4 rated 0.95 on coding-python; Mistral Medium 0.70; free-tier MiniMax 0.40) and provides calibrated profiles for endpoints without public benchmark data.

Stage 2: Reference Mapping. The Profiler treats the 48 canonical models as a reference corpus: each canonical model has both an observable feature vector $\mathbf{f}_M$ (from Stage 1) and a learned embedding $\mathbf{e}_M$ (from BilinearPhi's training). We fit two synthesizers on the 48 reference pairs:

- **Ridge linear regression**: $\mathbf{e}_M \approx \mathbf{W}_p \mathbf{f}_M + \mathbf{b}_p$, with ℓ_2 regularization to control the 80→256 underdetermination.
- **Weighted k-nearest-neighbors**: $\mathbf{e}_M \approx \sum_{j \in \mathcal{N}_k(M)} \alpha_j \mathbf{e}_j$, with softmax weights over feature-space cosine similarity.

The final synthesized embedding is the average of the two methods. A confidence score is attached based on cross-method agreement and feature-completeness.

Stage 3: Bulk Synthesis. Apply the fitted synthesizers to every production endpoint. The output is a JSON file mapping each endpoint id to its 256-dim embedding, confidence score, and provenance metadata. Production shadow router can now invoke BilinearPhi to score any endpoint, not just the canonical 48.

10.3 Validation

We validate the synthesis via leave-one-out cross-validation on the 48 reference models. For each canonical M: hold out M, refit both synthesizers on the remaining 47 pairs, predict $\hat{\mathbf{e}}_M$ from $\mathbf{f}_M$, compare to the true $\mathbf{e}_M$ via cosine similarity.

Table 20. Leave-one-out cosine similarity between synthesized and true canonical embeddings, with and without LLM-as-judge features.

Method	Structural features only (40d)	Structural + judge (80d)
Ridge linear	0.279	0.287
K-NN (k=5)	0.245	0.288
Ensemble	**0.274**	**0.304**

Two observations. First, the structural-only baseline already exceeds chance ($r > 0$ for all methods), but cosine ∼0.27 indicates the 40-dim feature vector is too low-rank to reconstruct the 256-dim embedding space precisely. Second, adding judge features improves cosine by 0.03, with cross-method agreement (confidence) rising from 0.65 to 0.88. The remaining gap is structural: 48 reference pairs is too few for a high-rank mapping. Approaching the cosine ∼0.65 quality observed for in-corpus models would require expanding the reference corpus to ∼200+ models, which in turn requires retraining the bilinear engine on a broader training set—a future-work direction.

10.4 Effect on the Three Views Comparison

Despite modest validation cosine, the synthesized embeddings substantially change the ğ7d Three Views result. Re-running the offline bilinear replay on the same 1,622 production prompts, now with Profiler

embeddings active for all 350 production endpoints:

Table 21. V1 scalar, BilinearPhi (48-model corpus only), and BilinearPhi + Profiler on the same 1,622 production prompts.

Metric	V1 scalar	Bilinear (48 only)	Bilinear + Profiler
Baseline (always-Opus, USD)	$101.30	$101.30	$101.30
Routed cost (USD)	$5.34	$92.22	$49.00
Cost reduction	**94.73%**	**7.94%**	**51.63%**
Agent picks (%)	25.4	0 (structural)	0 (structural)
Agreement with V1 actual (%)	—	0.0	0.0
Endpoints scorable	350	48	350

The Profiler-augmented bilinear achieves a 51.63% cost reduction versus the 7.94% baseline—a 6.5× improvement that comes purely from the embedding bootstrap, no retraining. Yet the comparison reveals an honest limitation: agreement with V1 remains 0%, and the bilinear still picks zero agents. This is because the synthesizers were fit on benchmark-derived pairs (RouterBench scores correlated with learned embeddings); they have no training signal for the routing patterns under which V1's online learner routes 25% of decisions to local agents.

10.5 Interpretation and Future Work

The Profiler shows that the bilinear vocabulary gap can be closed substantially via feature synthesis, but not eliminated. The two complementary directions that future work would pursue:

1. **Outcome-grounded fine-tuning.** The 402 production decisions where V1 routed to an agent constitute weak supervision: each decision implies a (prompt, agent, observed-quality) tuple. Fine-tuning the agent embeddings via gradient descent through the frozen bilinear matrices, with the production tuples as training signal, would teach the bilinear to recognize agent-favorable prompts. This is the next iteration's V3 direction.

2. **Expanded reference corpus.** Increasing the 48-model reference to the 145 production models with benchmark data would require retraining BilinearPhi on an expanded corpus, but would push leave-one-out cosine toward 0.5+ and likely recover meaningful agreement with V1.

The Profiler is shipped as agent code at `agents/profiler/` with a CLI for on-demand profiling of new endpoints. As new models are added to the production registry, the Profiler can be invoked to generate their embeddings without any retraining or downtime. This operationalizes the unification thesis at the system level: same equation, same scoring algebra, same online routing path, with bootstrap-quality embeddings for endpoints that fall outside the benchmark training distribution.

11 Related Work

LLM Routing and Cascading. FrugalGPT [Chen et al., 2023] pioneered cost-aware LLM selection through cascading strategies, achieving 50–90% cost reduction. Hybrid LLM [Ding et al., 2024] routes queries between small and large models based on predicted difficulty, reducing large-model calls by up to 40%. RouteLLM [Ong et al., 2024] trained pairwise classifiers for binary routing using preference data from LMSYS Chatbot Arena. AutoMix [Madaan et al., 2023] used self-verification to decide when to escalate from small to large models. Dekoninck et al. [2024] unified routing and cascading into a single framework, showing that iterative model selection with optional skipping outperforms pure routing or pure cascading by up to 14%. BEST-Route [Ding et al., 2025] extended this by jointly selecting both

model and number of responses based on query difficulty. RouterBench [Hu et al., 2024] and RouterEval [Huang et al., 2024] provide standardized evaluation frameworks. Most of these systems focus on LLM-to-LLM routing with additive or sequential scoring.

Learned Representations for Routing. A growing body of work learns query and model representations for routing decisions. RouteLLM's strongest baseline uses matrix factorization over preference data. RouterDC [Chen et al., 2024] trains a router via dual contrastive learning (sample-LLM and sample-sample losses) to match queries with the best-performing LLM from a candidate pool. EmbedLLM [Zhuang et al., 2024] learns compact vector embeddings of LLMs using an encoder-decoder approach, enabling performance forecasting across 112+ models. GraphRouter [Feng et al., 2024] constructs heterogeneous task-model graphs and uses edge prediction for routing. ICL-Router [Wang et al., 2025] uses in-context vectors to represent model capabilities, enabling integration of new models without retraining. Prompt-to-Leaderboard [Frick et al., 2025] trains an LLM to output Bradley-Terry coefficients from prompts, predicting human preferences for prompt-specific model ranking. ZeroRouter [Yan et al., 2026] creates a universal, model-agnostic latent space for query difficulty, enabling zero-shot onboarding of new models using only anchor queries. Our BilinearPhiEngine (Section 8) shares the learned-embedding approach with these systems. The bilinear scoring form $\phi(M,T) = e_M^\top W e_T$ is structurally equivalent to Factorization Machines [Rendle, 2010], which model pairwise interactions between features through low-rank factorization. The specific contribution is not the bilinear form itself, but integrating it as a drop-in replacement for hand-designed phi functions within the S(M,T) scoring architecture, preserving the framework's multiplicative gating and convergence properties while bypassing the measurement gap.

Multi-Agent and Heterogeneous Routing. Several recent systems extend routing beyond LLM-to-LLM selection. MoMA [Li et al., 2025] integrates LLM-as-a-judge with Mixture-of-Experts for unified routing across both models and agents. DAAO [Zhou et al., 2025] uses a VAE for difficulty estimation and dynamically generates query-specific multi-agent workflows. xRouter [Qian et al., 2025] frames routing as a tool-calling problem and trains end-to-end with reinforcement learning. Router-R1 [Zhang et al., 2025] teaches an LLM to interleave reasoning and routing actions for multi-round aggregation. S(M,T) also routes across heterogeneous endpoints (LLMs, agents, scripts, tool-use systems), but through a different mechanism: a single scoring equation with endpoint-type-specific gates rather than an LLM-based orchestrator.

Mixture of Experts. The Mixture-of-Experts architecture originated with Jacobs et al. [1991], who combined specialized expert networks through a learned gating function. Jordan and Jacobs [1994] extended this to hierarchical mixtures with EM training, establishing the multiplicative gating pattern that S(M,T) inherits. Sparse Mixture-of-Experts [Shazeer et al., 2017] scaled this to token-level routing within a single model. Switch Transformers [Fedus et al., 2022] simplified gating to top-1 expert selection. These approaches operate at the token level within a model, while S(M,T) operates at the query level across models. The Product-of-Experts framework [Hinton, 2002], which we use for the gate layer, was originally proposed for combining generative models.

Discrete Choice Theory. The S(M,T) equation form itself is not new; it has a fifty-year lineage in econometrics that an earlier version of this paper failed to credit. Under a log transform the score decomposes as $\log S(M,T) = \sum_j \log g_j(M,T) + \log\left(\sum_i w_i \phi_i(M,T)\right) - \beta(T) \cdot \mathbf{H}(M)$, and selecting endpoints by softmax over $\log S(M,T)$ (equivalently, sampling proportional to $S(M,T)$, as our Boltzmann selection does) is a conditional logit model [McFadden, 1974] over a constrained choice set. The correspondence is exact for two of the three components. The binary gate product implements choice-set

restriction: a failed gate sends $\log S(M,T)$ to $-\infty$, which is precisely the availability constraint $C \subseteq \mathscr{M}$ of discrete choice analysis [McFadden, 1974, Train, 2009]. The cost term contributes $-\beta(T) \cdot \mathbf{H}(M)$, a linear-in-parameters systematic utility component with task-conditional coefficients, McFadden's standard $V = \beta^\top x$ form with interaction terms. The compatibility term is the one nonstandard piece: $\log(\sum_i w_i \phi_i)$ is the log of a weighted sum rather than an additive-in-features utility. Via the identity $\log \sum_i w_i \phi_i = \log \sum_i \exp(\log w_i + \log \phi_i)$ it takes the inclusive-value (log-sum) functional form familiar from nested logit [Train, 2009], with unit nesting coefficient, although the ϕ_i are bounded features rather than utilities of actual sub-alternatives. Machine learning prior art reaches the same structure from the gating direction: mixture-of-experts gating [Jacobs et al., 1991] and sparsely-gated MoE [Shazeer et al., 2017] are softmax choice models with hard choice-set restriction. The deterministic $\arg\max_M S(M,T)$ rule is the zero-temperature limit of the same model. We therefore claim no novelty for the equation form: under log transform and softmax, S(M,T) is a constrained conditional logit with one log-sum compatibility regressor. What this paper contributes is the specific parameterization of that form for heterogeneous endpoint routing, the convergence, regret, and identifiability analysis of that parameterization (Section 4), and the empirical program (Sections 6 through 10).

Online Learning for Model Selection. The contextual bandit formulation for model selection has been explored by Agrawal and Goyal [2013] in the general setting. TI-UCB [Yu et al., 2024] adapts upper confidence bound methods for online model selection with time-increasing reward signals. BaRP [Hao et al., 2025] frames routing as a multi-objective contextual bandit conditioned on trade-off vectors, enabling performance-cost preference adjustment at inference without retraining. OmniRouter [Mei et al., 2025] models routing as constrained optimization via Lagrangian dual decomposition under budget constraints. Robbins-Monro stochastic approximation [Robbins and Monro, 1951] and convergence under Hajek conditions [Hajek, 1988] are well-established. Soft Actor-Critic [Haarnoja et al., 2018] introduced entropy-regularized learning for continuous control. We combine Robbins-Monro weight updates with SAC-style entropy decay for routing, with Duchi simplex projection [Duchi et al., 2008] to maintain valid weight distributions.

Benchmark Reliability. Concerns about benchmark contamination [Deng et al., 2023], evaluation methodology [Liang et al., 2022], and the gap between benchmark performance and real-world utility have been raised repeatedly. The measurement failures we document (Section 6) are instances of Goodhart's Law [Goodhart, 1975]: benchmark scores optimized as targets lose their value as routing signals. Manheim and Garrabrant [2019] categorize four variants of this effect, all present in our data. Our measurement gap analysis (Section 6) provides a systematic documentation of how these concerns specifically impact routing, demonstrating 0/14 successful measurement designs across 2.76M records.

What S(M,T) Adds. Given the breadth of recent routing work, we clarify the specific contributions of this paper. (1) *A fully analyzed parameterization, not a new equation form*: the three-part multiplicative form (gate products $\times$ weighted compatibility $\times$ Boltzmann cost) is, as the Discrete Choice Theory paragraph above details, a constrained conditional logit [McFadden, 1974, Train, 2009], and its individual components have well-established roots: the weighted compatibility layer in additive utility theory [Fishburn, 1967, Keeney and Raiffa, 1976], the gate layer in Product-of-Experts [Hinton, 2002] and MoE gating [Jacobs et al., 1991, Shazeer et al., 2017]. The contribution is the specific parameterization of that form for heterogeneous endpoint routing, carrying provable convergence ($O(\sqrt{KN \log N})$ regret), parameter identifiability, and adversarial robustness simultaneously. No prior routing system provides all three guarantees for the complete routing pipeline. (2) *The measurement gap as an empirical finding*: showing that hand-designed, interpretable scoring functions cannot be grounded in public

benchmark data due to metric incomparability, domain transfer breakdown, and dimensional collapse. This finding is independent of any particular routing architecture. (3) *Heterogeneous endpoint support via type-specific gates*: while MoMA and DAAO also route beyond LLMs, S(M,T) handles heterogeneous endpoints through the gate layer within a single equation rather than through an LLM orchestrator, preserving scoring interpretability and convergence properties. (4) *A learned resolution that preserves framework structure*: the BilinearPhiEngine replaces hand-designed phi functions with learned bilinear forms within the same S(M,T) architecture. The bilinear form itself builds on prior work in learned routing representations (RouterDC, EmbedLLM, GraphRouter). The contribution is demonstrating that this approach bypasses the measurement gap while retaining the framework's multiplicative gating and weight-learning machinery.

12 Discussion

12.1 The Interpretability-Performance Tradeoff

The most striking finding of this work is a tradeoff, not an impossibility. All 14 theoretical results (convergence, regret bounds, identifiability, orthogonality, adversarial robustness) verified at 85–95% confidence. Zero of 14 hand-designed measurement designs survived stress-testing. But the BilinearPhiEngine, using 16 learned bilinear forms on the same benchmark data, achieves 83.63% routing accuracy on RouterBench and AUC 0.8006 on RouteLLM.

The gap separates *interpretable* from *learned* routing. Hand-designed phi functions require explicit metric normalization, domain classification, and semantic mapping. Each of these steps introduces fragile assumptions that benchmark data cannot support. Learned bilinear forms sidestep all three by training end-to-end. The cost is that the 16 learned heads carry no semantic labels. We cannot say "head 3 captures reasoning depth" or "head 7 measures instruction adherence."

The V3 scaling ablation (Section 8.8) strengthens this finding. A 4.63× parameter increase with 2.2× more unique prompts and three architectural additions (task conditioning, cross-attention, metadata) uniformly degraded performance. The bottleneck is not model capacity but label quality: preference-based scores from heterogeneous datasets encode different semantics than benchmark accuracy, and naive data scaling amplifies this inconsistency rather than averaging it out.

This tradeoff has practical implications. For production routing where operators need to audit, debug, and explain routing decisions, the hand-designed phi functions with literature-informed seed weights and online learning remain the appropriate interface. For empirical validation that the weighted-phi architecture *can* achieve competitive accuracy, the bilinear engine provides the evidence. The two serve complementary roles.

12.2 Limitations

Seed Weight Subjectivity. The literature prior scores p_i (Section 7) were assigned by a single researcher surveying three papers. Different reviewers might assign different scores, shifting the seed weight vector. The Robbins-Monro learning procedure corrects for initialization bias over time (Proposition 4.9), but early routing decisions are sensitive to the prior.

Small Endpoint Set. The v0.1 router covers 13 endpoints. Production routing systems may handle hundreds. The $O(\sqrt{KN\log N})$ regret bound scales with K (phi dimensions, not endpoint count), but the gate evaluation cost scales linearly with the endpoint set size.

No A/B Testing. We have not compared S(M,T) routing decisions against human expert routing or competing algorithms on live traffic. The smoke tests (Table 6) verify correctness of the pipeline, not optimality of the routing policy.

Single-Turn Focus. The current framework scores each task independently. Multi-turn conversations, where context accumulates and the optimal endpoint may change mid-conversation, are not addressed.

Static Phi Definitions. The 16 phi functions are defined at design time. New capability dimensions (e.g., multimodal reasoning, tool-use orchestration) would require adding new phi functions and re-deriving seed weights.

12.3 Broader Impact

LLM routing affects cost and quality for anyone using language models. A good router reduces cost by directing simple tasks to cheap models while preserving quality on hard tasks. A bad router wastes money on overqualified models or produces poor results by underestimating task difficulty. The measurement gap finding has a specific implication: routing systems that claim benchmark-grounded accuracy should be scrutinized carefully. The normalization decisions, domain transfer assumptions, and contamination risks documented in Section 6 apply to all benchmark-based routing approaches, not just ours.

12.4 Future Directions

Routing-Specific Benchmarks. The field needs benchmarks designed for routing evaluation, not repurposed from model comparison. These would measure task-conditional, model-specific performance under deployment conditions, with controlled contamination and standardized metrics.

Multi-Turn Routing. Extending S(M,T) to multi-turn conversations requires a state variable tracking accumulated context, cost, and user satisfaction across turns. The scoring equation becomes $S(M, T, s_t)$ where s_t is the conversation state at turn t.

Federated Weight Learning. Multiple router deployments could share learned weights (after differential privacy protection) to accelerate convergence. The Robbins-Monro framework supports aggregated gradient estimates from distributed sources.

Bridging Interpretability and Performance. The BilinearPhiEngine (Section 8) demonstrates that learned phi functions achieve competitive routing accuracy. The open question is whether the learned bilinear heads can be *interpreted* post-hoc, perhaps through probing classifiers or attention-style analysis, to recover semantic labels. If so, the interpretability-performance tradeoff could be narrowed. The diversity regularization already encourages the heads to capture different aspects of model-task interaction; understanding *what* each head captures would connect the learned and hand-designed approaches.

Data Curation Over Model Scaling. The V3 ablation (Section 8.8) suggests that future routing improvements should prioritize training data curation over architectural scaling. Specifically: consistent scoring semantics across sources, rigorous model identity resolution, and filtering for label quality before increasing training set size. This echoes findings in the inverse scaling literature [McKenzie et al., 2023], where more capacity on noisy signals degrades performance.

13 Conclusion

We presented S(M,T), a unified scoring framework for routing queries to optimal endpoints. The framework combines multiplicative gating, weighted compatibility functions, and Boltzmann cost penalties into a single equation with provable convergence ($O(\sqrt{KN\log N})$ regret), parameter identifiability, and adversarial robustness. The equation form itself is a constrained conditional logit in the sense of McFadden [1974] (Section 11); the paper's contribution is the parameterization of that form for heterogeneous endpoint routing, the analysis attached to that parameterization, and the empirical findings below. It generalizes beyond LLM-to-LLM routing to cover agents, scripts, and tool-use systems through a common scoring interface.

Three empirical findings define the paper's contribution. First, the measurement gap: all 14 attempts to ground hand-designed phi functions in 2.76 million normalized benchmark records failed due to structural problems (metric incomparability, domain transfer breakdown, dimensional collapse). Second, the learned resolution: replacing the 16 hand-designed phis with 16 learned bilinear forms trained on 740,803 pairwise routing samples achieves 83.63% accuracy on RouterBench and AUC 0.8006 on RouteLLM, retaining 94.35% of always-strong quality at 50% cost reduction. Third, a scaling ablation: a 4.63× larger model (14.6M parameters) trained on expanded data with progressive architectural additions underperforms the compact V1 on every benchmark, establishing that data quality dominates model capacity for learned routing.

The measurement gap thus separates interpretable from learned routing, not theory from practice. Hand-designed phi functions that an operator can audit and explain cannot be grounded in public benchmark data. Learned bilinear forms that carry no semantic labels can. The tradeoff between interpretability and performance is the central tension this paper identifies, and narrowing that tradeoff is the most promising direction for future work.

The v0.1 router is operational, routing across 13 heterogeneous endpoints with literature-informed seed weights for the interpretable interface and bilinear heads for empirical validation.

AI Disclosure. AI tools developed by Anthropic were used to assist with writing, code development, and experimental analysis in the preparation of this work. All research direction, experimental design, and intellectual contributions are the author's own.

References

Shipra Agrawal and Navin Goyal. Thompson sampling for contextual bandits with linear payoffs. In *Proceedings of the 30th International Conference on Machine Learning*, pages 127–135, 2013.

Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256, 2002.

Donald T Campbell. Assessing the impact of planned social change. *Evaluation and Program Planning*, 2(1):67–90, 1979.

Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

Shuhao Chen, Weisen Jiang, Baijiong Lin, et al. RouterDC: Query-based router by dual contrastive learning for assembling large language models. *Advances in Neural Information Processing Systems*, 37, 2024.

Jasper Dekoninck, Maximilian Baader, and Martin Vechev. A unified approach to routing and cascading for LLMs. *arXiv preprint arXiv:2410.10347*, 2024. ICML 2025.

Chunyuan Deng, Yilun Zhao, Xiangru Tang, Mark Gerber, and Arman Cohan. Investigating data contamination in modern benchmarks for large language models. *arXiv preprint arXiv:2311.09783*, 2023.

Dujian Ding, Ankur Mallick, Chi Wang, et al. Hybrid LLM: Cost-efficient and quality-aware query routing. *arXiv preprint arXiv:2404.14618*, 2024. ICLR 2024.

Dujian Ding, Ankur Mallick, Shaokun Zhang, et al. BEST-Route: Adaptive LLM routing with test-time optimal compute. *arXiv preprint arXiv:2506.22716*, 2025. ICML 2025.

John Duchi, Shai Shalev-Shwartz, Yoram Singer, and Tushar Chandra. Efficient projections onto the ℓ_1-ball for learning in high dimensions. In *Proceedings of the 25th International Conference on Machine Learning*, pages 272–279, 2008.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

Tao Feng, Yanzhen Shen, and Jiaxuan You. GraphRouter: A graph-based router for LLM selections. *arXiv preprint arXiv:2410.03834*, 2024. ICLR 2025.

Peter C Fishburn. Additive utilities with incomplete product set: Applications to priorities and assignments. *Operations Research*, 15(3):537–542, 1967.

Evan Frick, Connor Chen, Joseph Tennyson, Tianle Li, et al. Prompt-to-leaderboard. *arXiv preprint arXiv:2502.14855*, 2025.

Charles A E Goodhart. Problems of monetary management: The U.K. experience. In *Papers in Monetary Economics*. Reserve Bank of Australia, Sydney, 1975.

Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning*, pages 1861–1870, 2018.

Bruce Hajek. Cooling schedules for optimal annealing. *Mathematics of Operations Research*, 13(2): 311–329, 1988.

Botao Hao et al. Learning to route LLMs from bandit feedback: One policy, many trade-offs. *arXiv preprint arXiv:2510.07429*, 2025.

Geoffrey E Hinton. Products of experts. In *Proceedings of the 9th International Conference on Artificial Neural Networks*, pages 1–6, 1999.

Geoffrey E Hinton. Training products of experts by minimizing contrastive divergence. *Neural Computation*, 14(8):1771–1800, 2002.

Qitian Hu et al. RouterBench: A benchmark for multi-LLM routing system. *arXiv preprint arXiv:2403.12031*, 2024.

Sicheng Huang, Qitian Hu, et al. RouterEval: A comprehensive evaluation framework for LLM routers. *arXiv preprint arXiv:2407.12606*, 2024.

Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991.

Michael I Jordan and Robert A Jacobs. Hierarchical mixtures of experts and the EM algorithm. *Neural Computation*, 6(2):181–214, 1994.

Ralph L Keeney and Howard Raiffa. *Decisions with Multiple Objectives: Preferences and Value Tradeoffs*. John Wiley & Sons, New York, 1976.

Yehuda Koren, Robert Bell, and Chris Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009.

Harold J Kushner and G George Yin. *Stochastic Approximation and Recursive Algorithms and Applications*. Springer, 2nd edition, 2003.

Zixuan Li et al. Towards generalized routing: Model and agent orchestration for adaptive and efficient inference. *arXiv preprint arXiv:2509.07571*, 2025.

Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narang, et al. Holistic evaluation of language models. *arXiv preprint arXiv:2211.09110*, 2022.

Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

Aman Madaan, Pranjal Aggarwal, Prakhar Anand, Srividya Pranavi Potluri, Swaroop Mishra Gupta, Dheeraj Rajagopal, Vladlen Koltun, and Yiming Yang. AutoMix: Automatically mixing language models. *arXiv preprint arXiv:2310.12963*, 2023.

David Manheim and Scott Garrabrant. Categorizing variants of Goodhart's law. *arXiv preprint arXiv:1803.04585*, 2019.

Daniel McFadden. Conditional logit analysis of qualitative choice behavior. *Frontiers in Econometrics*, pages 105–142, 1974.

Ian R McKenzie, Alexander Lyzhov, Michael Pieler, Alicia Parrish, Aaron Mueller, Ameya Prabhu, Euan McLean, Aaron Kirtland, Alexis Ross, Alisa Liu, et al. Inverse scaling: When bigger isn't better. *Transactions on Machine Learning Research*, 2023.

Kai Mei, Wujiang Xu, Minghao Guo, et al. OmniRouter: Budget and performance controllable multi-LLM routing. *arXiv preprint arXiv:2502.20576*, 2025.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E Gonzalez, M Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*, 2024.

Cheng Qian, Zuxin Liu, Shirley Kokane, et al. xRouter: Training cost-aware LLMs orchestration system via reinforcement learning. *arXiv preprint arXiv:2510.08439*, 2025.

Steffen Rendle. Factorization machines. In *Proceedings of the 10th IEEE International Conference on Data Mining*, pages 995–1000. IEEE, 2010.

Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951.

Thomas J Rothenberg. Identification in parametric models. *Econometrica*, 39(3):577–591, 1971.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Marafino, Le Davis, Quoc Le, and Geoffrey Hinton. Out-

rageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.

Kenneth E Train. *Discrete Choice Methods with Simulation*. Cambridge University Press, Cambridge, 2nd edition, 2009.

Eric Walter and Luc Pronzato. *Identification of Parametric Models from Experimental Data*. Springer, 1997.

Chenxu Wang, Hao Li, Yiqun Zhang, et al. ICL-Router: In-context learned model representations for LLM routing. *arXiv preprint arXiv:2510.09719*, 2025. AAAI 2026.

Cheng Yan et al. Breaking model lock-in: Cost-efficient zero-shot LLM routing via a universal latent space. *arXiv preprint arXiv:2601.06220*, 2026.

Guangning Yu et al. Which LLM to play? convergence-aware online model selection with time-increasing bandits. In *Proceedings of the ACM Web Conference*, 2024.

Haozhen Zhang et al. Router-R1: Teaching LLMs multi-round routing and aggregation via reinforcement learning. *arXiv preprint arXiv:2506.09033*, 2025.

Chen Zhou et al. Difficulty-aware agentic orchestration for query-specific multi-agent workflows. *arXiv preprint arXiv:2509.11079*, 2025.

Richard Zhuang, Tianhao Wu, Zhaojin Wen, et al. EmbedLLM: Learning compact representations of large language models. *arXiv preprint arXiv:2410.02223*, 2024. ICLR 2025.